



Università di Napoli Federico II

Corso di Laurea in Informatica

ARCHITETTURA DEGLI ELABORATORI: A.A.2019-2020

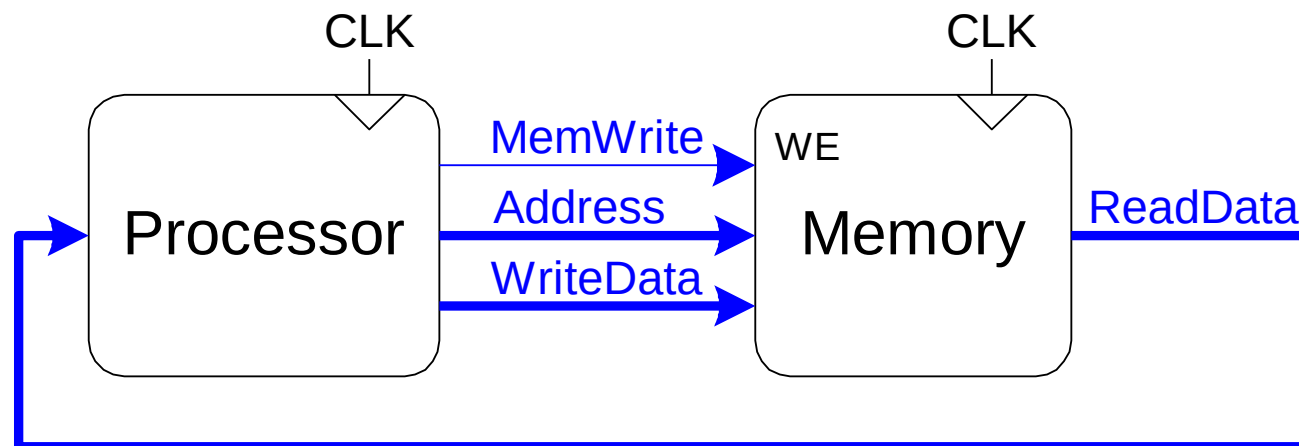
Memorie

Prof. Luigi Sauro – gruppo1 (A-G)

Memorie vs CPU

Nell'architettura Von Neuman il canale di comunicazione tra la CPU e la memoria è il punto critico (collo di bottiglia) del sistema.

La tecnologia consente di realizzare CPU sempre più veloci e memorie sempre più grandi ma la velocità di accesso delle memorie non cresce così rapidamente come la velocità della CPU.



Memorie vs CPU

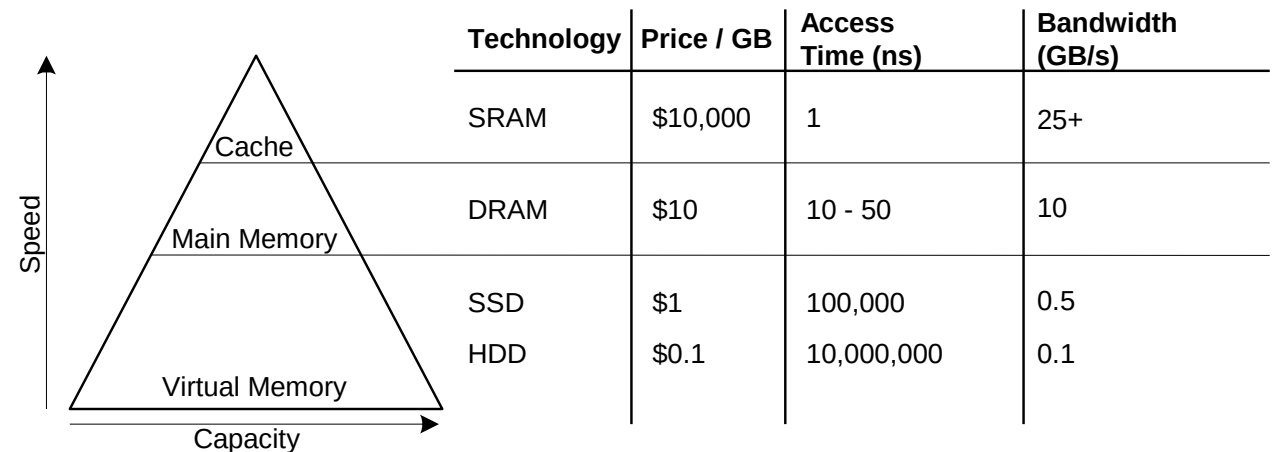
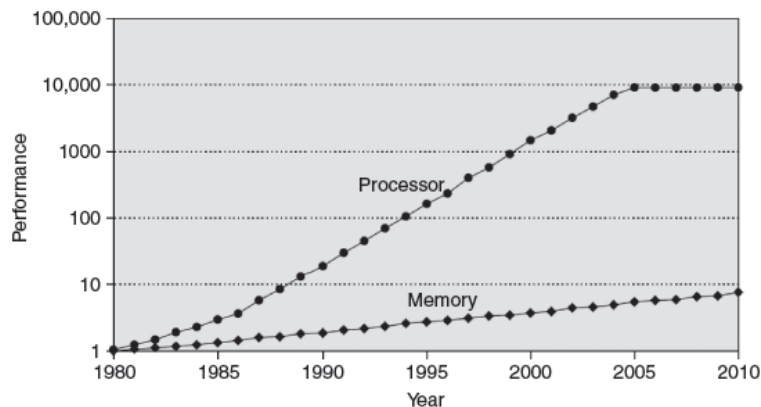
Storicamente le **CPU** sono sempre state più veloci delle **memorie**.

Al giorno d'oggi siamo in grado di produrre delle memorie veloci quanto una CPU moderna, il reale problema è dato da:

- queste memorie hanno un costo elevatissimo.
- le memorie dovrebbero essere piazzate in gran parte sugli stessi chip delle CPU, il che non è possibile.

Per ovviare a questo problema, gli ingegneri ricorrono ad uno schema chiamato “a gerarchia di **memoria**” in cui si combinano:

- una quantità molto **piccola** di memoria estremamente **veloce** (la cache)
- una quantità molto **grande** di memoria **lenta**.



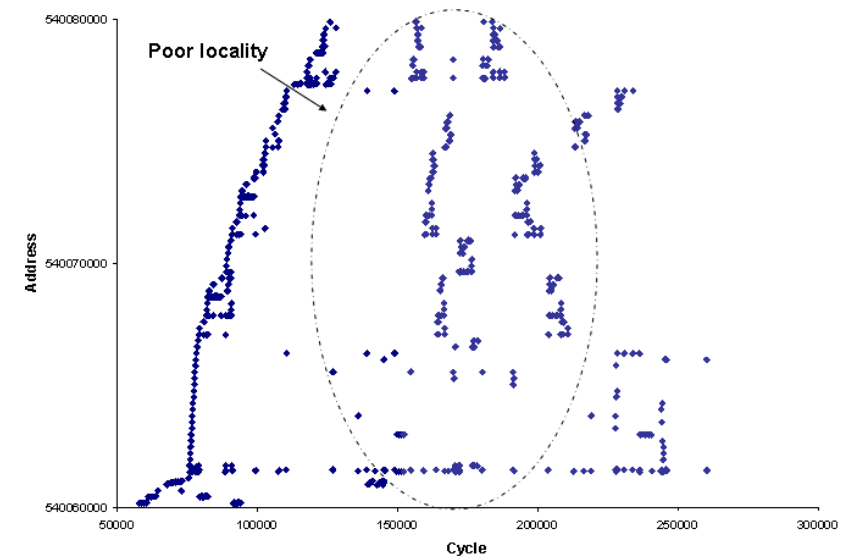
Inquadramento del Problema

La maggior parte del tempo di esecuzione di una CPU è, di solito, impegnato da procedure **in cui vengono eseguite ripetutamente le stesse istruzioni**.

In realtà, non è importante conoscere lo schema dettagliato della sequenza di istruzioni, il punto chiave è che molte istruzioni in aree ben localizzate del programma vengono eseguite ripetutamente in un determinato periodo, e si accede al resto del programma relativamente di rado.

Questa proprietà viene chiamata **località dei riferimenti**. Si manifesta in due modi: **località temporale** e **località spaziale**.

- La **località temporale** rappresenta la probabilità che un'istruzione eseguita di recente venga eseguita nuovamente, entro breve tempo.
- La **località spaziale** rappresenta invece la probabilità che istruzioni vicine ad un'istruzione eseguita di recente (dove la vicinanza è espressa in termini di indirizzi delle istruzioni) siano anch'esse eseguite nel prossimo futuro.



Utilità di avere una cache memory

La velocità con cui la memoria risponde alle richieste di istruzioni e dati della CPU ha un peso determinante sulle prestazioni di un sistema.

Se tra la memoria principale e la CPU si potesse **interporre una memoria molto veloce, contenente le parti di programma e di dati che, volta per volta, interessano l'elaborazione**, il tempo totale di esecuzione verrebbe ridotto in modo significativo.

Questa è esattamente la funzione della "**cache memory**" che in inglese significa letteralmente "**schermo della memoria**".

Si tratta quindi di un elemento che inserito tra CPU e memoria principale impedisce alla prima di "**vedere**" i tempi di risposta reali della memoria.

La struttura della CPU, infatti, non viene influenzata dalla presenza o meno di una **cache memory**. L'interfaccia verso la memoria applica il "**protocollo**" di trasferimento **dalla** memoria o **verso** la memoria ignorando la presenza di una struttura intermedia.

Agli albori di questa tecnica, infatti, la cache memory era solo un accessorio a pagamento.

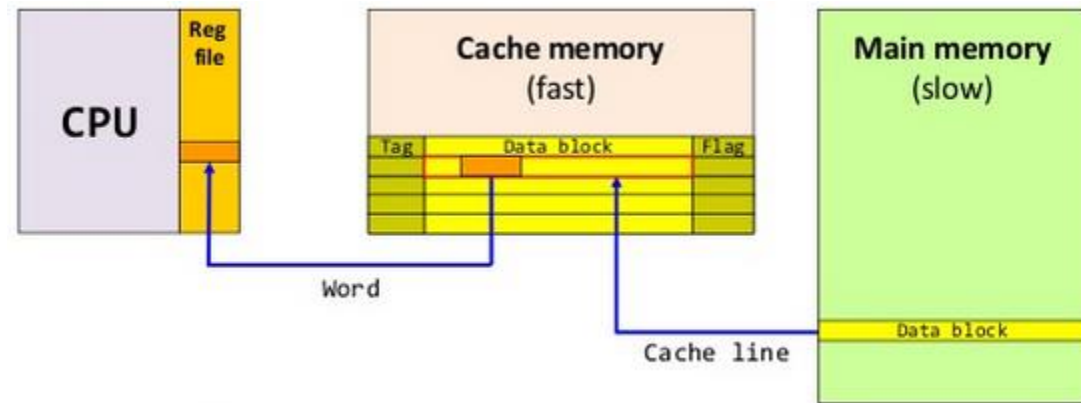
Una traduzione possibile in italiano del concetto è "**memoria tampone**".

Principi di funzionamento

Concettualmente, le operazioni svolte da una memoria cache sono molto semplici. I circuiti di controllo della memoria cache sono progettati per avvantaggiarsi della proprietà della località dei riferimenti.

- L'aspetto temporale di questa proprietà suggerisce di portare un elemento (istruzioni o dati) nella cache quando viene richiesto per la prima volta, in modo tale che rimanga a disposizione nel caso di una nuova richiesta.

- L'aspetto spaziale suggerisce che, invece di portare dalla memoria principale alla cache un elemento alla volta, è conveniente portare un insieme di elementi che risiedono in indirizzi adiacenti.



Si farà riferimento al termine **blocco** per indicare un insieme di indirizzi contigui di una qualche dimensione.

Un altro termine utilizzato di frequente per indicare un blocco della cache è **linea di cache**.

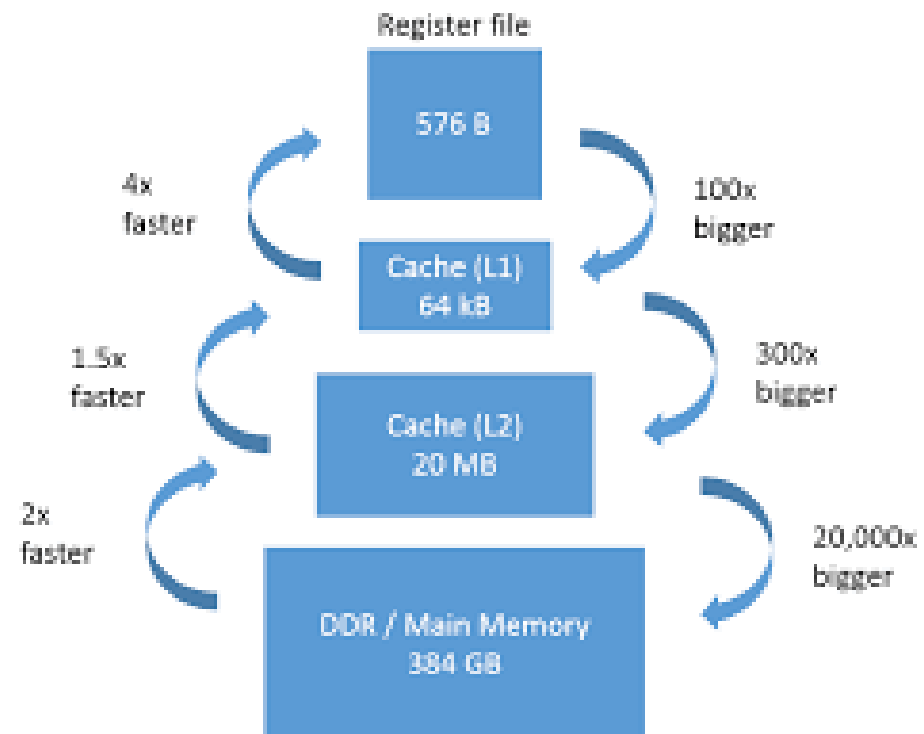
Dimensione della cache

Quando si riceve una richiesta di lettura fatta dalla CPU, il contenuto di un blocco di parole di memoria contenenti la locazione specificata viene trasferito nella cache, una o più parole alla volta.

In seguito, ogniqualvolta il programma fa riferimento a una di queste locazioni del blocco, i valori desiderati vengono letti direttamente dalla cache.

Affinché la cache svolga efficacemente il suo compito deve avere tempi di accesso molto più brevi di quelli della memoria principale e ciò come vedremo impone che sia piccola.

Se è piccola non potrà che contenere una frazione ridotta delle istruzioni ed i dati della memoria principale.



Tecniche di gestione

La corrispondenza tra i blocchi della memoria principale e quelli della cache è specificata dalla funzione di ***posizionamento (mapping)***.

Quando la cache è piena e si fa riferimento a una parola di memoria (istruzione o dato) che non è presente nella cache, l'hardware di controllo della cache deve decidere quale blocco della cache debba essere rimosso per far spazio al nuovo blocco, che contiene la parola a cui si fa riferimento.

L'insieme di regole in base alle quali viene fatta questa scelta costituisce ***l'algoritmo di sostituzione***.

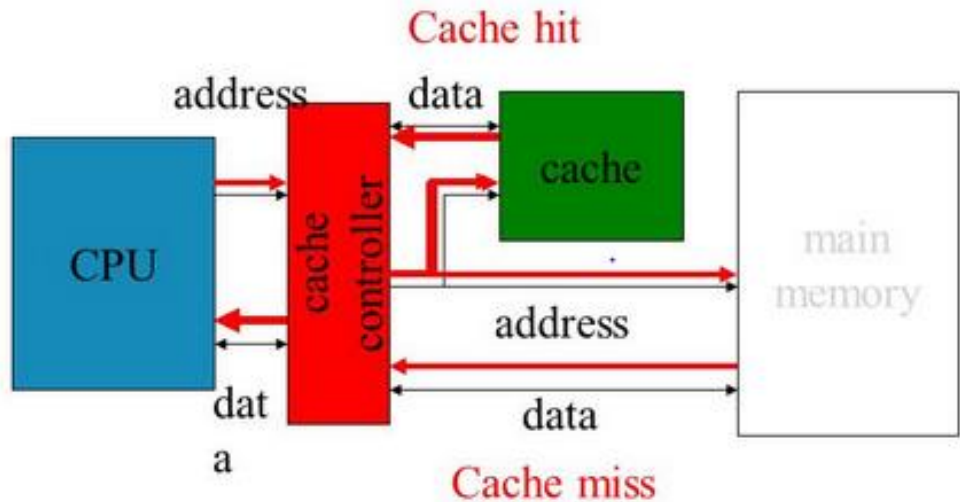
Tecniche di gestione

La struttura dei sistemi è, di solito, organizzata in modo che la presenza di una cache non influenzi il funzionamento della CPU.

Quest'ultima, infatti, nell'esecuzione del programma **effettua le richieste di lettura e scrittura utilizzando gli indirizzi delle locazioni nella memoria principale.**

E' la logica di controllo della cache che si fa carico di determinare se la parola richiesta è presente o meno nella cache.

Se è presente, viene effettuata l'operazione di lettura o scrittura della locazione di memoria appropriata. In questo caso si dice che **l'accesso in lettura o in scrittura ha avuto successo** (*read hit* o *write hit*).



Possibili gestioni di un “cache hit”

Se l'accesso alla cache ha avuto successo bisogna distinguere due tipi di situazioni:

Operazione di lettura, la memoria principale non viene coinvolta

Operazione di scrittura, il sistema può procedere in due modi:

- Nel primo (***write-through***), la locazione della cache e quella della memoria principale **vengono entrambe aggiornate**.
 - Il ***write-through*** è più semplice, ma implica inutili operazioni di scrittura nella memoria principale, se una parola viene aggiornata più volte durante il periodo in cui risiede nella cache.
- Nel secondo (***write-back***) si aggiorna **soltanto la locazione della memoria cache**, segnandola come aggiornata con un **bit di modifica** o ***dirty***. La locazione della memoria principale viene aggiornata in seguito, quando il blocco contenente la parola marcata deve essere rimosso dalla memoria cache per far posto a un nuovo blocco.
 - Anche il protocollo ***write-back*** può causare inutili scritture nella memoria principale, visto che, quando si procede con la scrittura nella memoria principale di un blocco, **tutte le parole del blocco vengono scritte**, anche se solo una delle parole del blocco della cache è stata modificata.

Gestione di un “read miss”

Quando la parola indirizzata durante un'operazione di lettura non è presente nella cache si dice che **l'accesso in lettura è fallito (*read miss*)**.

Il blocco di parole contenente la parola richiesta deve essere copiato dalla memoria principale nella memoria cache.

Due sono le possibilità:

Dopo aver caricato l'intero blocco nella memoria cache, la parola richiesta viene inviata alla CPU.
In alternativa, è possibile inviare immediatamente la parola alla CPU, non appena la si legge dalla memoria principale.

Quest'ultimo approccio, chiamato ***load-through***, o anche ***early restart***, riduce in qualche modo il tempo di attesa della CPU, a discapito di una maggiore complessità del circuito di controllo della cache.

Gestione di un “write miss”

Durante un'operazione di scrittura, se la parola indirizzata non è nella cache si dice che **l'accesso in scrittura è fallito (*write miss*)**.

Anche in questo caso ci sono le due alternative, se si utilizza un protocollo ***write-through***, le informazioni vengono scritte direttamente nella memoria principale, nel caso invece di un protocollo ***write-back***, il blocco, contenente la parola indirizzata, viene prima caricato nella cache, poi viene sovrascritto con le nuove informazioni.

Prestazioni

Hit Rate = # hits / # memory accesses

= 1 – Miss Rate

Miss Rate = # misses / # memory accesses

= 1 – Hit Rate

Average memory access time (AMAT): average time for processor to access data

$$\text{AMAT} = t_{\text{cache}} + MR_{\text{cache}}[t_{MM} + MR_{MM}(t_{VM})]$$

Prestazioni

- Un programma ha 2,000 loads e stores
- 1,250 di questi dati sono presenti nella cache
- Il resto è fornito da altri livelli nella gerarchia di memoria
- **Quale è l'hit e il miss rate per la cache?**

$$\text{Hit Rate} = 1250/2000 = 0.625$$

$$\text{Miss Rate} = 750/2000 = 0.375 = 1 - \text{Hit Rate}$$

Prestazioni

- Assumi che un processore ha 2 livelli di memoria: cache e main memory
- $t_{\text{cache}} = 1 \text{ cycle}$, $t_{MM} = 100 \text{ cycles}$
- Qual'è l'AMAT dati gli hit e miss rate precedenti?

$$\begin{aligned} \text{AMAT} &= t_{\text{cache}} + MR_{\text{cache}}(t_{MM}) \\ &= [1 + 0.375(100)] \text{ cycles} \\ &= 38.5 \text{ cycles} \end{aligned}$$

Terminologia Cache

- **Capacity (C):**

- Numero di byte nella cache

- **Block size (b):**

- Grandezza di un blocco, ovvero numero di bytes che sono trascritti nella cache per volta

- **Number of blocks ($B = C/b$):**

- Numero di blocchi nella cache

- **Degree of associativity (N):**

- Numero di blocchi in un set

- **Number of sets ($S = B/N$):**

- Ogni indirizzo di memoria è mappato in esattamente un set della cache

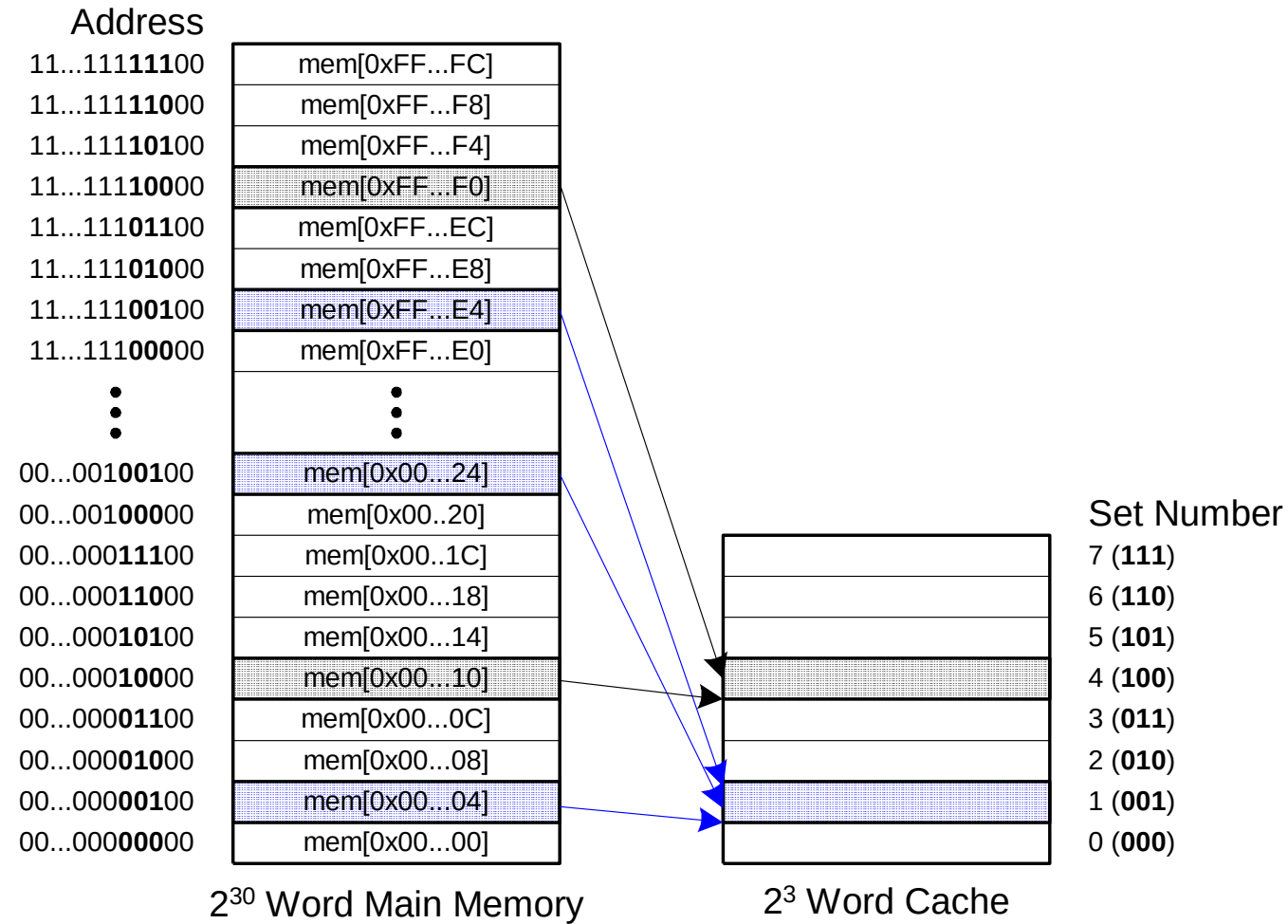
Tipologie di memorie cache

- La memoria Cache è organizzata in S set
 - Ogni indirizzo di memoria mappa in esattamente un unico set
 - Diverse categorie di memoria cache dipendono dal numero di blocchi che un set contiene:
 - **Direct mapped:** 1 blocco per ogni set
 - **N -way set associative:** N blocchi per set
 - **Fully associative:** tutti i blocchi di una cache sono in un unico set
 - Esaminiamo ogni tipologia con una cache con:
 - Capacity ($C = 8$ words)
 - Block size ($b = 1$ word)
 - Quindi, il numero di blocchi è ($B = 8$)
- Ovviamente una memoria del genere è ridicolmente piccola, serve solo per scopi didattici*

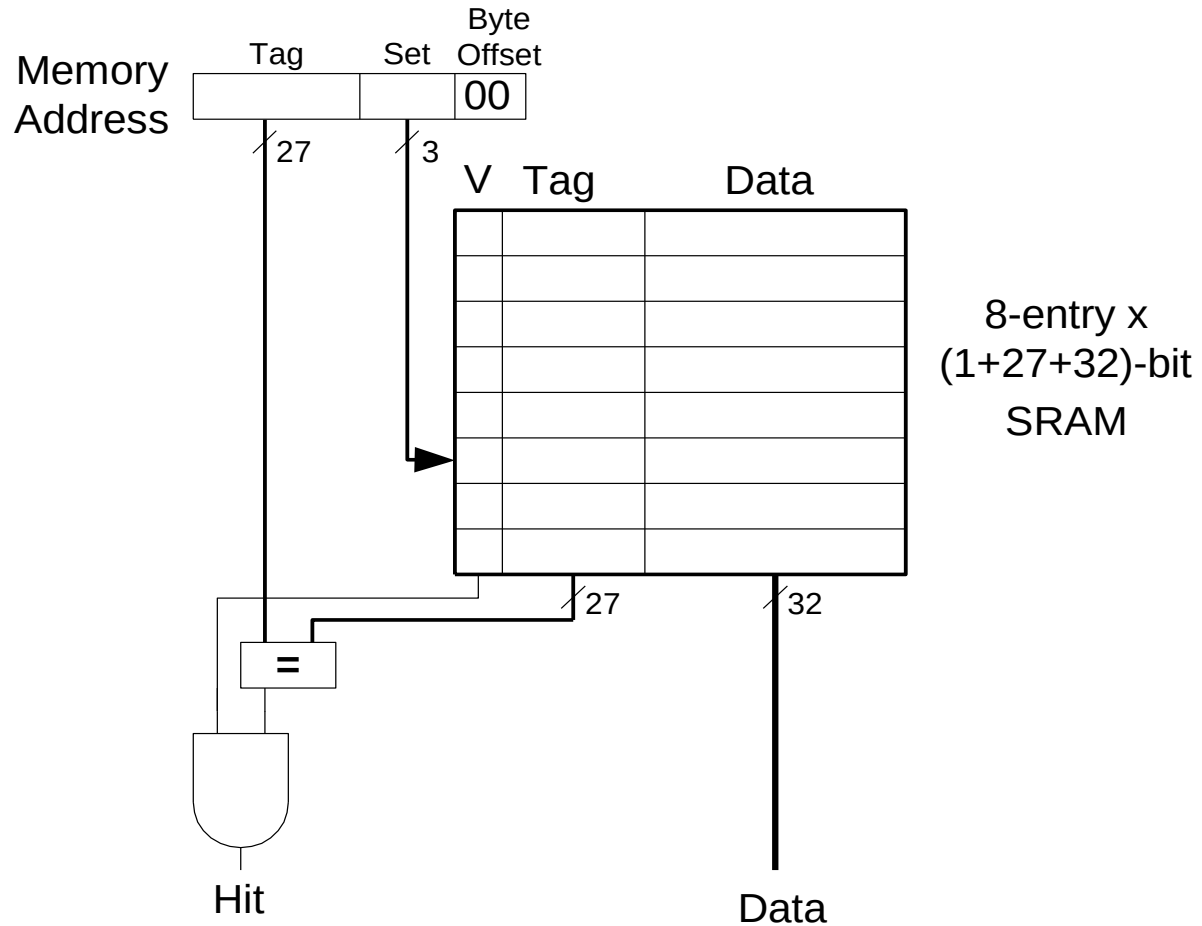
- Capacity: C
- Block size: b
- Number of blocks in cache: $B = C/b$
- Number of blocks in a set: N
- Number of sets: $S = B/N$

Organization	Number of Ways (N)	Number of Sets ($S = B/N$)
Direct Mapped	1	B
N-Way Set Associative	$1 < N < B$	B / N
Fully Associative	B	1

Direct Mapped Cache

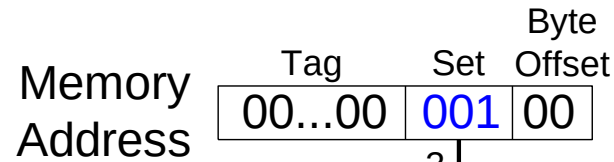


Direct Mapped Cache Hardware



ARM Assembly Code

```
        MOV R0, #5
        MOV R1, #0
LOOP    CMP R0, #0
        BEQ DONE
        LDR R2, [R1, #4]
        LDR R3, [R1, #12]
        LDR R4, [R1, #8]
        SUB R0, R0, #1
        B    LOOP
DONE
```



V	Tag	Data	
0			Set 7 (111)
0			Set 6 (110)
0			Set 5 (101)
0			Set 4 (100)
1	00...00	mem[0x00...0C]	Set 3 (011)
1	00...00	mem[0x00...08]	Set 2 (010)
1	00...00	mem[0x00...04]	Set 1 (001)
0			Set 0 (000)

Miss Rate = 3/15
= 20%

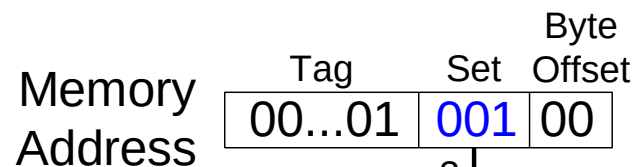
Miss obbligatori

ARM Assembly Code

```

        MOV R0, #5
        MOV R1, #0
LOOP    CMP R0, #0
        BEQ DONE
        LDR R2, [R1, #0x4]
        LDR R3, [R1, #0x24]
        SUB R0, R0, #1
        B    LOOP
DONE

```



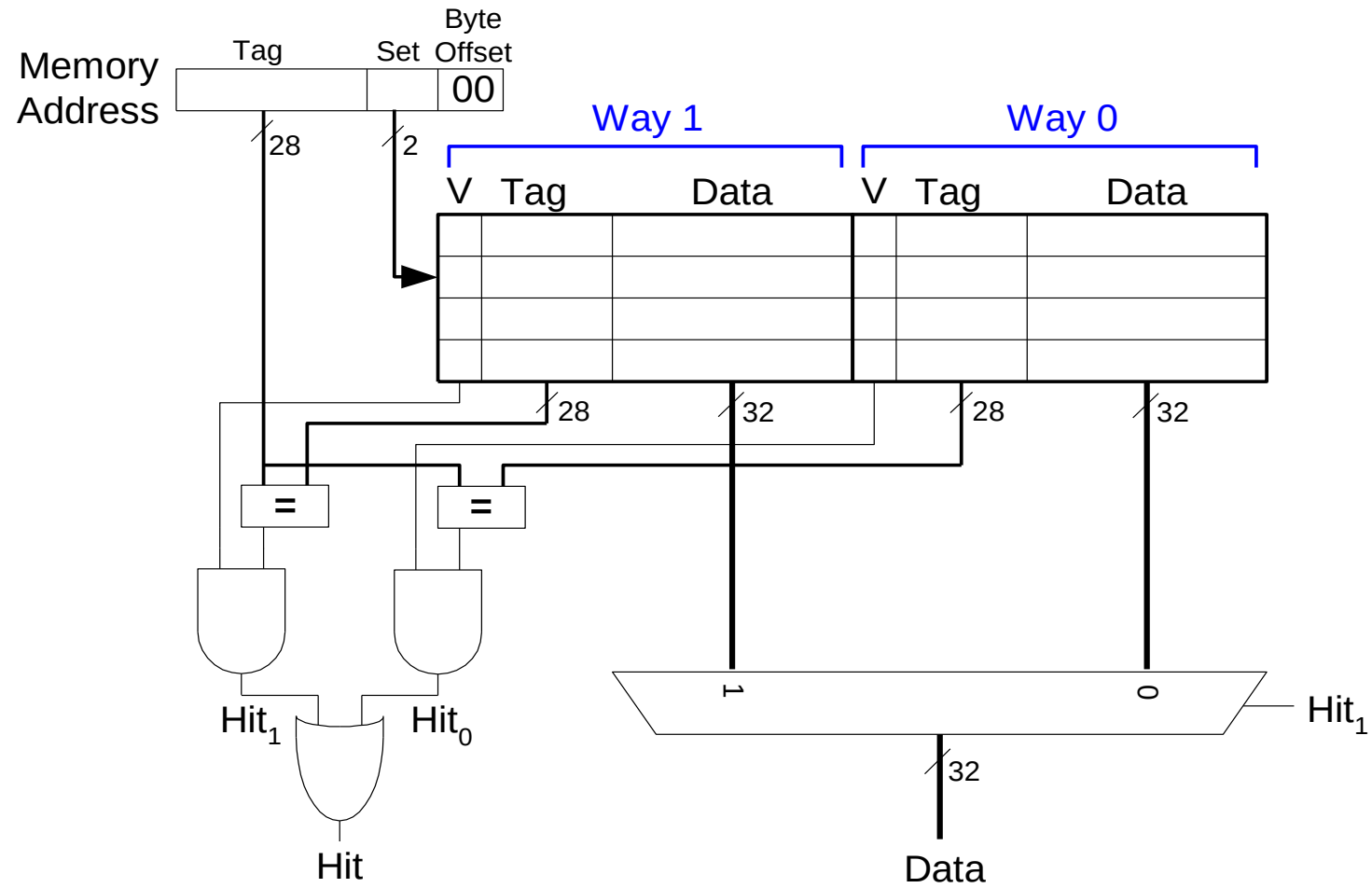
3

V	Tag	Data	
0			Set 7 (111)
0			Set 6 (110)
0			Set 5 (101)
0			Set 4 (100)
0			Set 3 (011)
0			Set 2 (010)
1	00...00	mem[0x00...04] mem[0x00...24]	Set 1 (001)
0			Set 0 (000)

Miss Rate = 10/10
= 100%

Miss di conflitto

N-Way Set Associative Cache



ARM Assembly Code

```
MOV R0, #5
MOV R1, #0
LOOP  CMP R0, 0
      BEQ DONE
      LDR R2, [R1, #0x4]
      LDR R3, [R1, #0x24]
      SUB R0, R0, #1
      B   LOOP
DONE
```

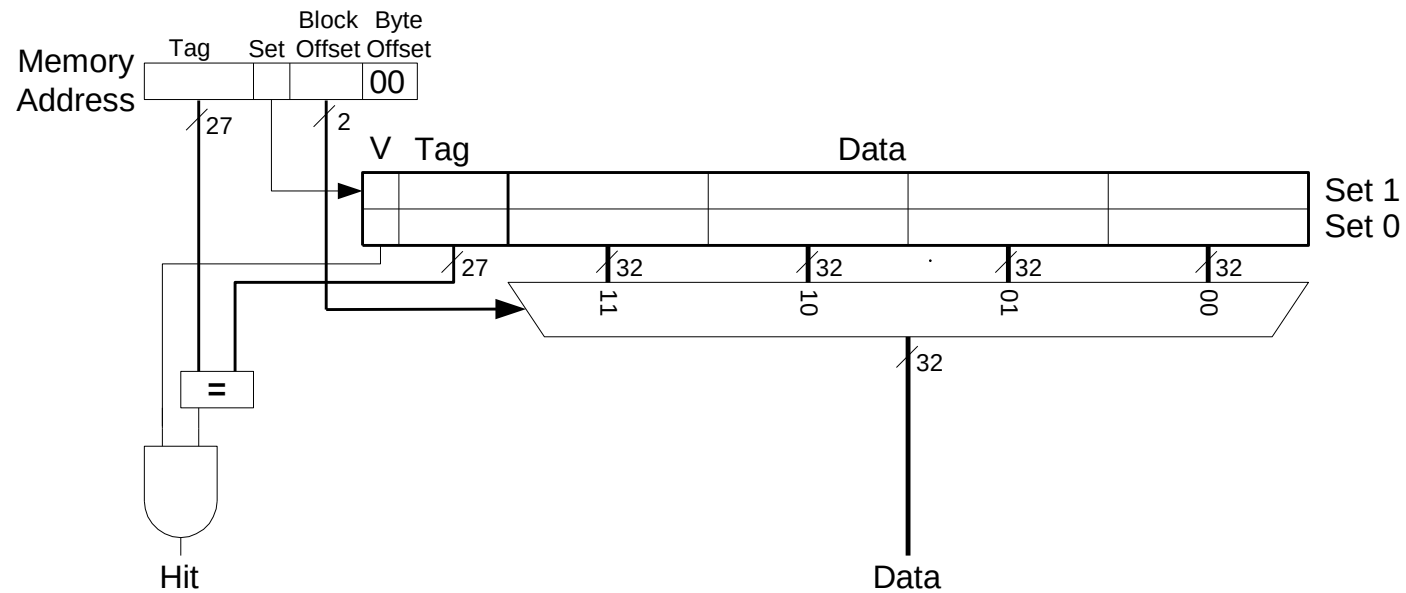
**Miss Rate = 2/10
= 20%**

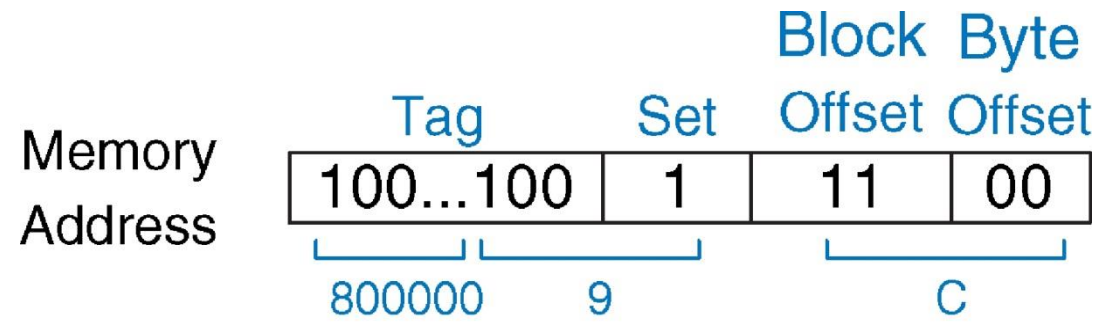
**L'associatività riduce i miss di
conflitto**

Way 1			Way 0			
V	Tag	Data	V	Tag	Data	
0			0			Set 3
0			0			Set 2
1	00...10	mem[0x00...24]	1	00...00	mem[0x00...04]	Set 1
0			0			Set 0

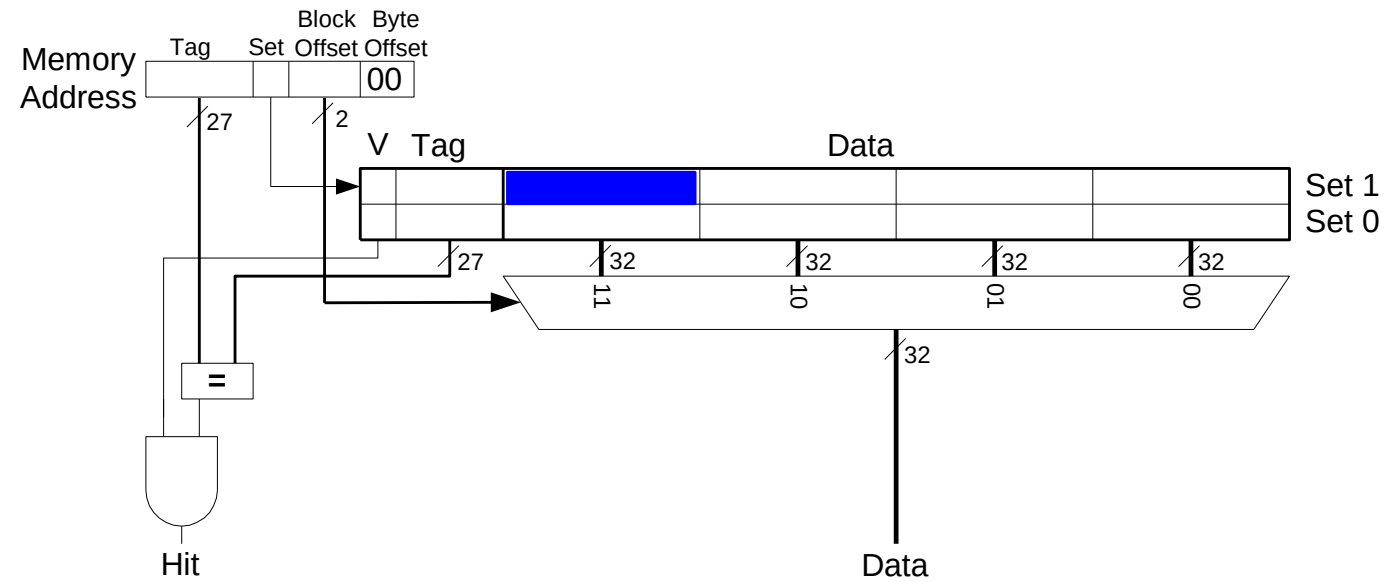
Come sfruttare la località spaziale?

- Incrementare il block size:
 - Block size, **$b = 4$ words**
 - $C = 8$ words
 - Direct mapped (1 block per set)
 - Number of blocks, **$B = 2$** ($C/b = 8/4 = 2$)





© 2007 Elsevier, Inc. All rights reserved

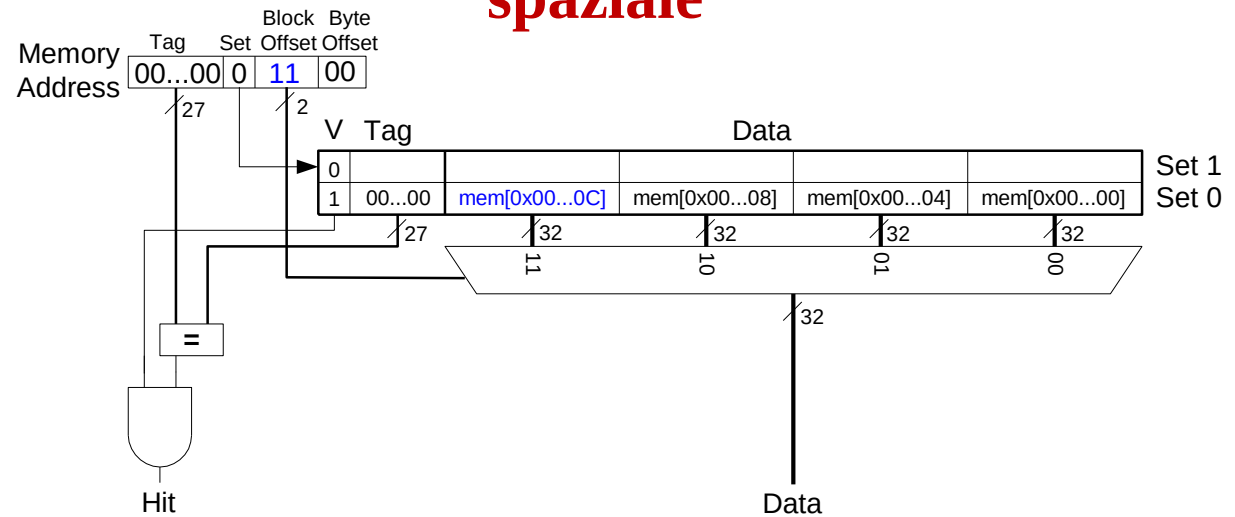


ARM assembly code

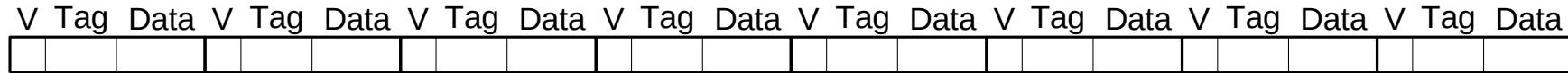
```
        MOV R0, #5
        MOV R1, #0
LOOP    CMP R0, 0
        BEQ DONE
        LDR R2, [R1, #4]
        LDR R3, [R1, #12]
        LDR R4, [R1, #8]
        SUB R0, R0, #1
        B    LOOP
DONE
```

Miss Rate = 1/15
= 6.67%

**Blocchi più grandi riducono
i miss obbligatori
attraverso la località
spaziale**



Fully Associative Cache



- Consente la massima flessibilità nella scelta della locazione nella cache in cui posizionare un blocco di memoria
- Lo spazio della cache può essere utilizzato in modo più efficiente e questo comporta una riduzione dei miss di conflitto
- Il costo della ricerca di un blocco in cache è notevole, mentre nel caso di indirizzamento diretto è praticamente nullo
 - Una operazione di questo tipo si chiama ricerca associativa o per contenuto, in quanto corrisponde all'operazione di ricerca di un item in un elenco
- E' più costosa da costruire

Il bit di validità

- Un ulteriore bit di controllo, chiamato *bit di validità*, è necessario per ogni blocco.
- **Questo bit indica se il blocco contiene o meno dati validi.**
- Non deve però essere confuso con il *bit di modifica*, menzionato in precedenza, il bit di modifica, che indica se il blocco è stato modificato o meno durante il periodo in cui è stato nella cache, serve soltanto nei sistemi che non fanno uso del metodo *write-through*.

I bit di validità dei blocchi che si trovano nella cache vengono tutti posti a 0 nell'istante iniziale di accensione del sistema e, in seguito, ogni volta che nella memoria principale vengono caricati programmi e dati nuovi dal disco.

Il Direct Memory Access - DMA

I trasferimenti dal disco alla memoria principale vengono effettuati mediante un meccanismo detto di DMA (Direct Memory Access). Questa sigla specifica che durante il trasferimento la CPU non interviene, perché è la logica di controllo del disco che gestisce le linee indirizzo e le linee dati, fornendo i segnali di controllo necessari.

Come vedremo successivamente, in una operazione di questo genere vengono trasferite grandi quantità di istruzioni e dati organizzati in entità dette “***pagine***” ciascuna delle quali di solito contiene migliaia di ***locazioni*** e quindi centinaia di ***blocchi***.

Normalmente, le pagine di programma e quelle di dati vanno nella memoria centrale senza coinvolgere la cache.

Gestione del bit di validità

Il bit di validità di un certo blocco della cache viene posto a “1” la prima volta che esso è chiamato a contenere istruzioni o dati trasferiti dalla memoria principale; poi, ogni volta che le locazioni di un blocco della memoria principale vengono interessate da un trasferimento di nuove istruzioni o di nuovi dati dal disco, viene effettuato un controllo per determinare se qualcuno dei blocchi che stanno per essere sovrascritti fossero o meno presenti nella cache.

Se nella cahe c'è una copia del blocco, il suo bit di validità viene posto a “0”; in tal modo, si garantisce che nella cache non ci siano dati obsoleti.

Il bit di validità a “0” candida immediatamente il blocco ad essere sovrascritto.

Svuotamento (flush) della cache

Una situazione dello stesso tipo si determina quando viene effettuato un trasferimento tramite DMA dalla memoria al disco e la cache utilizza un protocollo write-back.

In questo caso, i dati nella memoria potrebbero non riflettere i cambiamenti che possono essere stati fatti sulla copia dei dati presenti nella cache.

Una possibile soluzione a tale problema consiste nello ***svuotamento (flush)*** della cache forzando i dati con il bit di modifica attivo a essere ricopiati nella memoria principale prima che venga effettuato il trasferimento tramite DMA.

“Coerenza” della cache

Il sistema operativo è in grado di fare tutto tutte le operazioni necessarie in modo molto efficiente, senza peggiorare significativamente le prestazioni, visto che le operazioni di ingresso/uscita di scrittura non sono molto frequenti.

Questa necessità di garantire che due diverse entità (i sottosistemi della CPU e del DMA nel caso presente) utilizzino la stessa copia dei dati viene chiamata problema della **coerenza della cache**.

Algoritmi di sostituzione (1 / 2)

- In una cache ad indirizzamento diretto, la posizione di ogni blocco è predefinita, quindi non esiste alcuna strategia di sostituzione.
- Nelle memorie cache *associative* e *set-associative*, esiste invece una certa flessibilità.

Quando un nuovo blocco deve essere portato nella cache, e tutte le posizioni che potrebbe occupare contengono dati validi, il controllore della cache deve decidere quale blocco, tra quelli esistenti, sovrascrivere.

Questa è una decisione importante, perché la scelta potrebbe essere determinante per le prestazioni del sistema.

Algoritmi di sostituzione (2 / 2)

In generale, l'obiettivo è quello di **mantenere nella cache quei blocchi che hanno una maggior possibilità di essere nuovamente utilizzati nel prossimo futuro**. ma, non è facile prevedere i blocchi a cui si farà riferimento.

Una possibile strategia è di tener presente che la ***località dei riferimenti*** suggerisce che i blocchi a cui c'è stato accesso di recente hanno elevata probabilità di essere utilizzati nuovamente entro breve tempo.

Sostituzione con algoritmo LRU

Quando bisogna eliminare dalla cache un blocco, è **sensato sovrascrivere quello a cui non si accede da più tempo**. Tale blocco prende il nome di blocco *utilizzato meno di recente* (*Least Recently Used, LRU*), e la tecnica si chiama *algoritmo di sostituzione LRU*.

Per utilizzare l'algoritmo LRU, il controllore della cache deve mantenere traccia di tutti gli accessi ai blocchi mentre l'elaborazione prosegue.

Il blocco LRU di una memoria cache *set-associativa* con insiemi di quattro blocchi, può, ad esempio, essere indicato da un contatore di 2 bit associato a ciascun blocco, il cui contenuto sia opportunamente gestito secondo un protocollo automatico

Gestione del contatore di accessi

Quando si ha un successo nell'accesso al blocco (*hit*):

- il contatore di quel blocco viene posto a 0. I contatori con valori originariamente inferiori a quelli del blocco a cui si accede, vengono incrementati di uno, mentre tutti gli altri rimangono invariati.

Quando, invece, l'accesso fallisce (*miss*) si aprono due possibilità:

- l'insieme non è pieno, il contatore associato al nuovo blocco caricato dalla memoria principale viene posto a 0 e il valore di tutti gli altri contatori viene incrementato di 1.
- l'insieme è pieno, si rimuove il blocco, il cui contatore ha valore 3, e si pone il nuovo blocco al suo posto, mettendo il contatore a 0. Gli altri tre contatori dell'insieme vengono incrementati di un'unità.

E' facile verificare che i valori dei contatori dei blocchi occupati sono sempre diversi.

Alternative all'algoritmo LRU

L'algoritmo LRU è stato utilizzato ampiamente, con buone prestazioni, in numerosi schemi di accesso. Ma in alcune situazioni, per esempio nel caso di accessi sequenziali a un vettore di elementi che è leggermente troppo grande per poter stare tutto nella cache, le prestazioni sono molto scadenti

Le prestazioni dell'algoritmo LRU possono essere migliorate introducendo una piccola dose di casualità nella scelta del blocco da sostituire.

Sono stati proposti molti altri algoritmi di sostituzione più o meno complessi, che richiedono talvolta anche notevoli complicazioni hardware, ma l'algoritmo più semplice, e spesso anche piuttosto efficace, consiste nello **scegliere in modo completamente casuale il blocco da sostituire.**

Tipi di Miss

- **Compulsory:** primo accesso ad un dato
- **Capacity:** la cache è troppo piccola per contenere tutti i dati di interesse
- **Conflict:** due dati sono indirizzati nella stessa locazione nella cache

Miss penalty: il tempo necessario per recuperare un blocco da un livello più basso nella gerarchia delle memorie

ARM Assembly Code

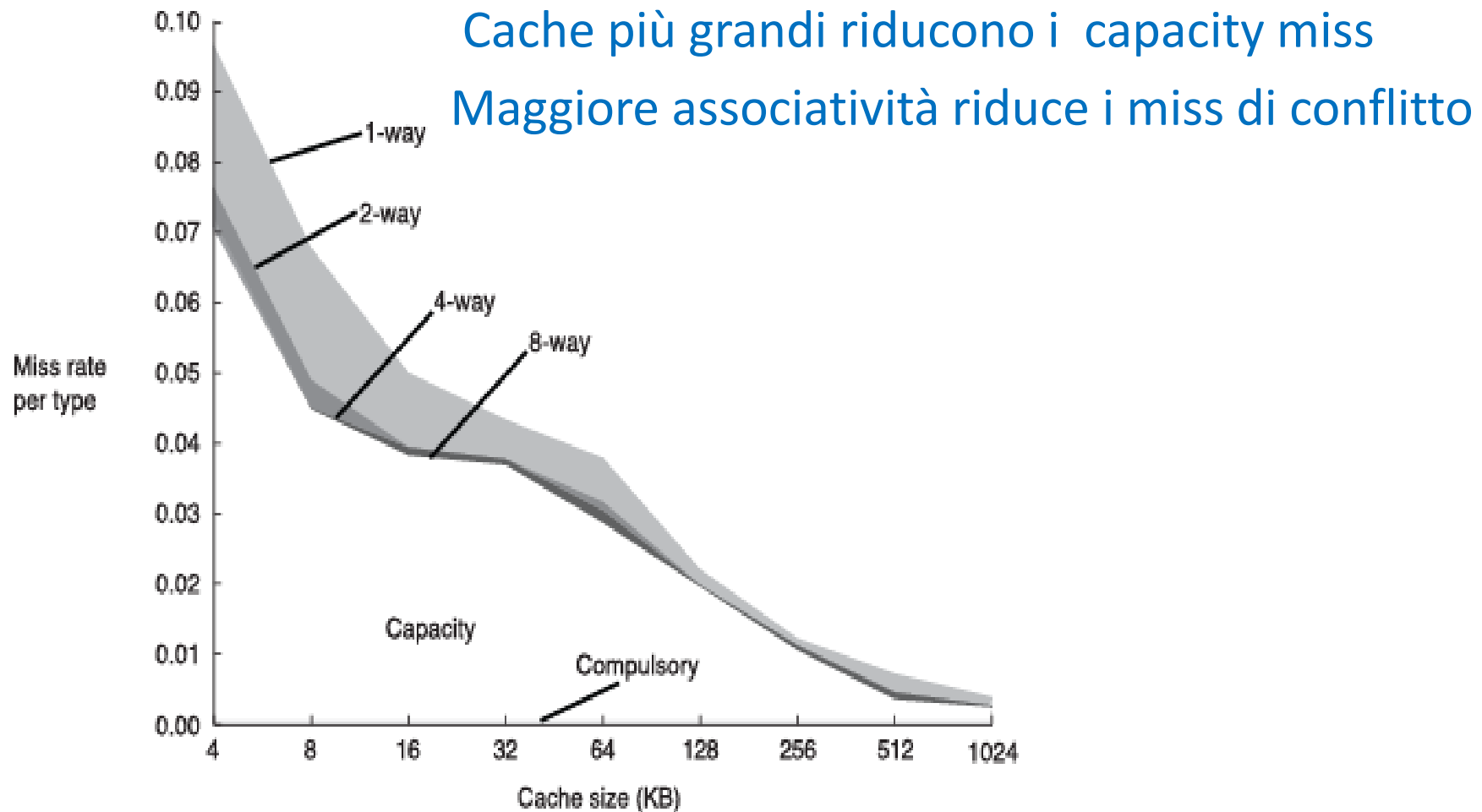
```
MOV R0, #0
LDR R1, [R0, #4]
LDR R2, [R0, #0x24]
LDR R3, [R0, #0x54]
```

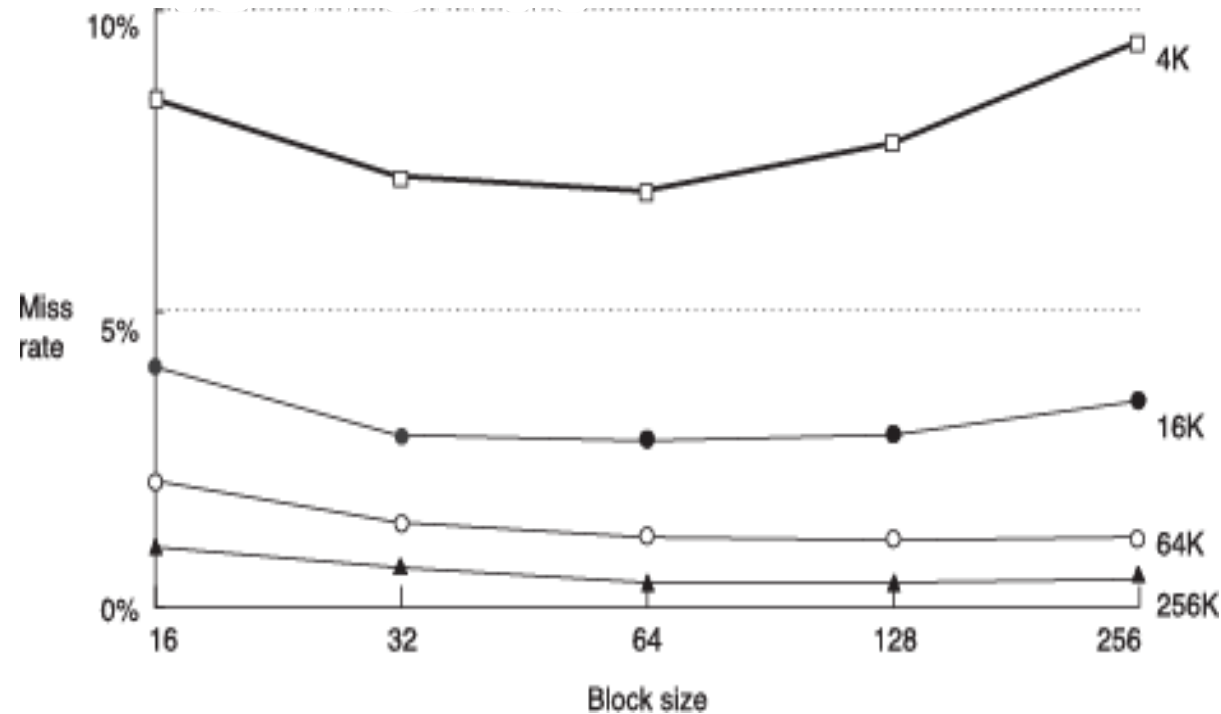
Way 1				Way 0			
V	U	Tag	Data	V	Tag	Data	
0	0			0			Set 3 (11)
0	0			0			Set 2 (10)
1	0	00...010	mem[0x00...24]	1	00...000	mem[0x00...04]	Set 1 (01)
0	0			0			Set 0 (00)

(a)

Way 1				Way 0			
V	U	Tag	Data	V	Tag	Data	
0	0			0			Set 3 (11)
0	0			0			Set 2 (10)
1	1	00...010	mem[0x00...24]	1	00...101	mem[0x00...54]	Set 1 (01)
0	0			0			Set 0 (00)

(b)





- Blocchi più grandi riducono i miss obbligatori sfruttando la località spaziale
- Data una cache di dimensioni fissate, all'aumentare del block size il numero di set decrementa ($S \times N \times b = C$). Quindi aumentano i miss di conflitto

Frazionamento della memoria in moduli

E utile che i trasferimenti da/verso le unità più veloci possano essere effettuati alla stessa frequenza dell'unità più veloce.

Ciò non può avvenire se si accede nello stesso modo all'unità più lenta e a quella più veloce.

Tale risultato può invece essere raggiunto se si sfrutta il parallelismo nell'organizzazione dell'unità più lenta.

Un modo efficiente per introdurre il parallelismo nella gestione della memoria principale consiste nell'impiego di un'organizzazione *interallacciata*.

Se la memoria principale di un calcolatore è strutturata come una collezione di moduli fisicamente separati, ognuno con il proprio registro degli indirizzi (Address Register, AR) e registro dei dati (Data Register, DR), le operazioni di accesso alla memoria possono procedere contemporaneamente in più di un modulo.

Così, si può incrementare la frequenza aggregata di trasmissione delle parole da o verso il sistema di memoria principale.

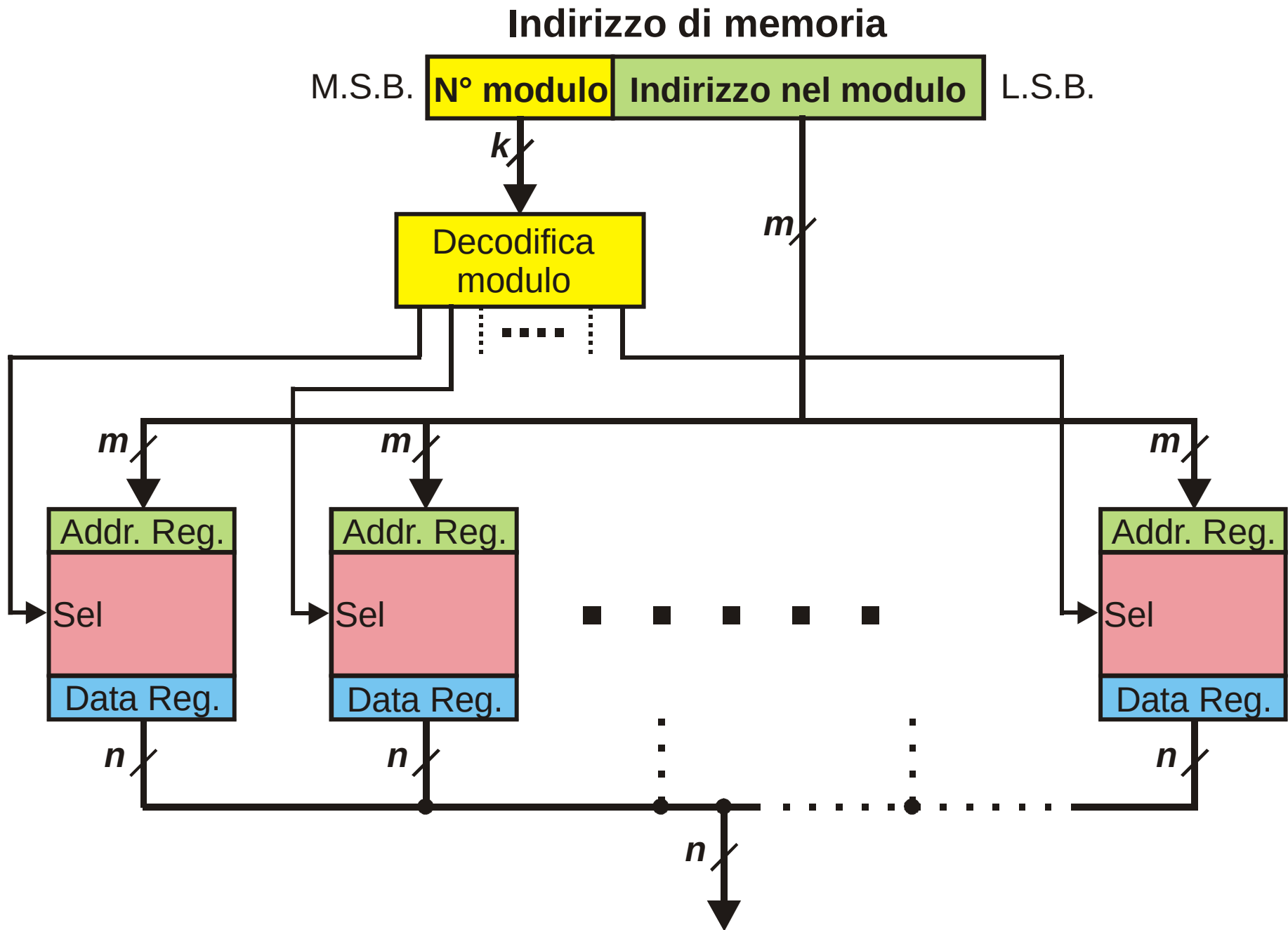
Distribuzione degli indirizzi nei moduli

Il modo in cui gli indirizzi individuali sono distribuiti attraverso i moduli è importante per stabilire il numero medio di moduli che può essere tenuto occupato mentre le elaborazioni procedono.

Esistono due metodi di distribuzione degli indirizzi.

Nel primo, l'indirizzo di memoria generato dalla CPU viene decodificato nel seguente modo:

I k bit più significativi indicano uno degli n moduli, e gli m bit meno significativi indicano una particolare parola all'interno del modulo. Quando si accede a locazioni consecutive, come accade quando si copia un blocco di dati nella cache, viene coinvolto soltanto un modulo. Nello stesso tempo, tuttavia, dispositivi con possibilità di accesso diretto alla memoria (DMA) possono aver accesso alle informazioni contenute in altri moduli della memoria.



Distribuzione degli indirizzi nei moduli

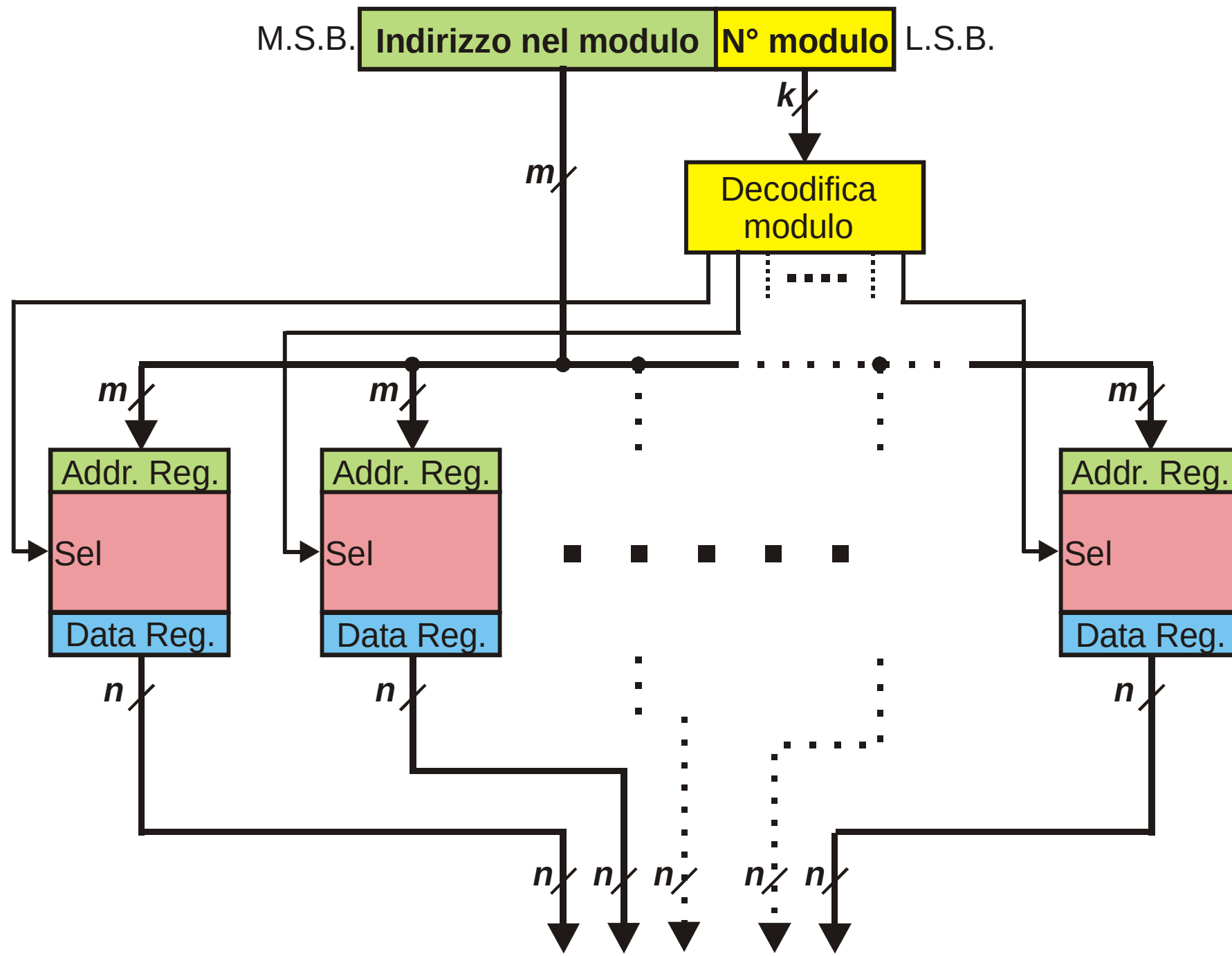
Il secondo e più efficiente modo di indirizzare i moduli è chiamato ***interallacciamento della memoria***.

I k bit meno significativi dell'indirizzo di memoria selezionano un modulo, mentre gli m bit più significativi indicano una locazione all'interno del modulo. In questo modo, indirizzi consecutivi sono posizionati in moduli successivi; così facendo, ogni componente del sistema che faccia richiesta di accesso a locazioni di memoria consecutive può tenere occupati vari moduli in un qualsiasi istante. Ciò consente di ottenere sia accessi a un blocco di dati più veloci, sia una più elevata media complessiva di utilizzo della memoria.

Per realizzare la struttura interallacciata, devono esserci $2k$ moduli.

In caso contrario, ci sono degli stati vuoti associati a locazioni inesistenti nello spazio di indirizzamento della memoria.

Indirizzo di memoria

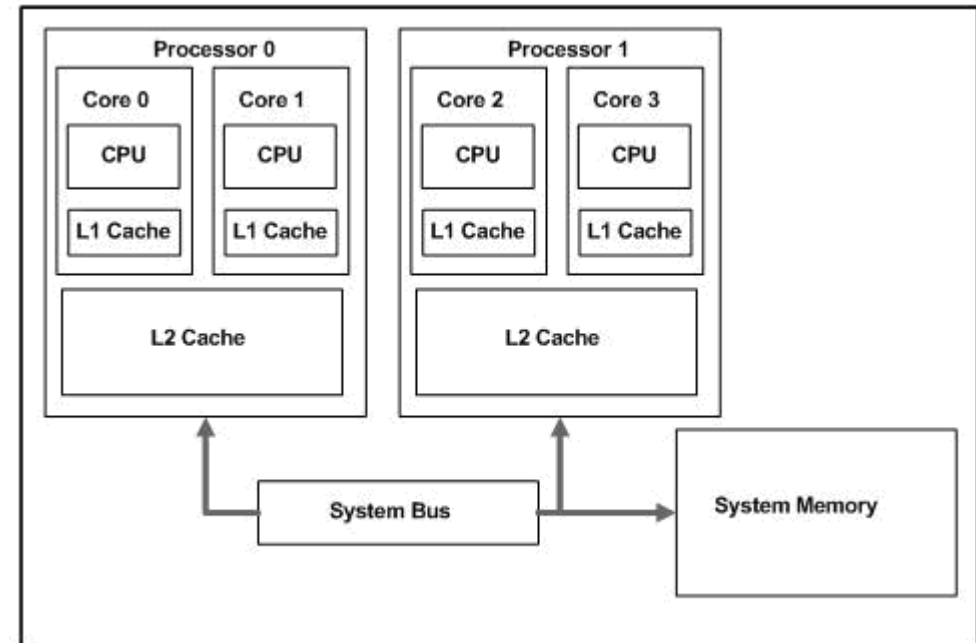


Cache multi livello

- Oggigiorno la memoria cache costituisce una gerarchia di 2 o 3 livelli: L1, L2, L3
- Vi possono essere due tipologie di cache multi livello: inclusiva o esclusiva
- Inclusiva: L2 contiene i dati presenti in L1 (similmente L2 è contenuta in L3)
 - L2 contiene sia i dati correnti di L1, sia quelli che erano presenti in precedenza e sono stati sovrascritti mediante l'algoritmo di sostituzione (copy back)
 - Per mantenere la coerenza L1 ha una politica di write-through verso L2, e così L2 verso L3
 - Miss in L1 e hit in L2: il blocco relativo viene copiato da L2 a L1 (in caso di sostituzione un dato va da L2 a L1 e un'altro da L1 a L2)
 - Miss in L1 e L2: il blocco dalla main memory viene copiato sia in L1 che L2
- Esclusiva: L2 contiene solo i dati di copy back da L1 (victim cache)
 - La memoria complessiva della cache è data dalla somma delle capacità dei vari livelli, in realtà nelle moderne architetture la capacità di L2 è un ordine di grandezza rispetto a quella di L1, e così fra L3 e L2, il guadagno rispetto alle cache inclusive non è molto significativo).
 - Miss in L1 e hit in L2: Le linee di cache di L1 e L2 vengono scambiati fra loro, cioè la linea di cache di L1 viene memorizzata in L2 e la linea di L2 in L1.
 - Miss in L1 e L2: Il dato letto dalla memoria è memorizzato direttamente in L1 e la linea di cache rimpiazzata di L1 (*victim data*) è trasferita in L2 rimpiazzando un'altra linea di cache secondo la politica di rimpiazzo usata.

Cache condivise

- Nelle architetture multi core L2 o L3 possono essere condivisi fra più core di uno stesso processore, questo consente:
- Un uso più efficiente del livello condiviso
 - Se un core è inattivo, allora l'altro può utilizzare per se il livello condiviso
- Nella programmazione multi-threading consente di utilizzare in maniera più efficiente dati condivisi
 - Un core può eseguire un pre-/post-processing di dati forniti dall'altro core
- Riduce il front-side bus traffic
 - Un unico livello condiviso che si interfaccia con la main memory



La Memoria Virtuale

Agli albori dell'informatica la memoria centrale di un computer raramente superava le 4k parole ed i sistemi operativi erano, a dir poco, essenziali.

A quell'epoca, chiunque si disponesse a scrivere un programma doveva prima informarsi di quale fosse la disponibilità di spazio nella memoria centrale del sistema da adoperare, al netto delle necessità della CPU e della “parte residente” del sistema operativo.

Fatto il conteggio doveva scrivere un programma fatto di pezzi da mandare in esecuzione uno per volta (**overlay, sovrapposizione**), registrando i risultati parziali in gruppi di memoria i cui indirizzi fossero noti anche ai successivi pezzi di programma.

Questi li prelevavano per usarli ed, a loro volta registravano i risultati in locazioni prefissate.

Senza questa procedura il programma non poteva essere eseguito.

La Memoria Virtuale

Tutto questo è oggi completamente ignoto agli utilizzatori di computer.

Il fatto che talvolta siano indicati dei requisiti minimi del sistema per usare un certo programma, è un semplice suggerimento per ottenere dal programma un comportamento ragionevole, senza lunghissimi tempi di attesa tra due operazioni successive.

Oggi è assolutamente normale che programmi che tra occupazione diretta ed indiretta avrebbero bisogno di 25-30 Mbyte girino su macchine con memoria centrale da 16 Mbyte, grazie alla tecnica della “**memoria virtuale**”, inventata da un gruppo di ricercatori inglesi di Manchester nel 1961 ed adottata dall'IBM negli anni '70.

La tecnica si avvale di strutture hardware che è fondamentale che siano nel circuito integrato della CPU e ciò spiega perché essa è apparsa nei sistemi basati su microprocessori “single chip” relativamente di recente.

La gestione è affidata al sistema operativo

In estrema sintesi la tecnica della *memoria virtuale* prevede che il sistema operativo assegni una parte degli spazi liberi nella memoria centrale ai segmenti di un programma istallato su disco.

Il sistema operativo seguirà criteri prefissati, assegnando uno spazio compreso tra un minimo ed un massimo, in funzione della dimensione del programma.

Gli spazi potranno essere anche non continui.

Il risultato è che programmi piccoli saranno trasferiti interamente ma, naturalmente, in funzione della mappa di occupazione corrente potranno essere dislocati in maniera frammentata in qualsiasi parte della memoria.

La memoria è gestita dinamicamente

Programmi di grandi dimensioni avranno a disposizione spazi insufficienti a contenerli completamente e saranno soggetti a continue operazioni di “**swapping**”, cioè di sostituzione dinamica dei loro segmenti nello spazio di memoria centrale assegnato.

L'operatore può imporre al sistema operativo di dedicare più spazio ad un programma, ma se esso usa una gran parte del campo d'indirizzamento del processore, non potrà in ogni caso essere completamente trasferito in memoria.

La tecnica di gestione degli spazi ha molti punti in comune con il funzionamento di una memoria cache.

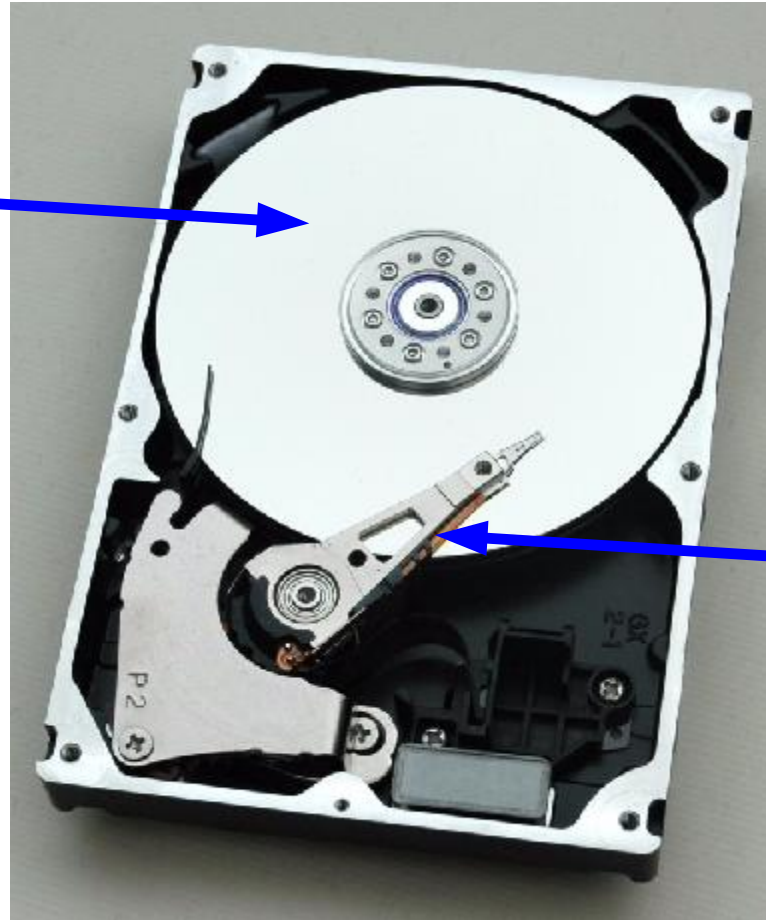
Anche in questo caso, infatti, se gli spazi a disposizione nella memoria principale sono saturati, e il programma accede ad una locazione di memoria che non corrisponde ad un indirizzo esistente nella memoria centrale, si deve ricorrere ad un algoritmo di sostituzione.

Dimensioni della pagina di memoria

La parte di programma contenente l'indirizzo richiesto viene prelevato dal disco (trasferimento in DMA) e sostituita in memoria ad una delle parti che vi risiedevano.

Poiché il tempo di accesso al disco costituisce una penalità di fallimento molto più pesante, in termini di tempo rispetto a quella di una cache, l'entità minima da trasferire è presa molto più grande del blocco della cache ed è una *pagina di memoria* di solito compresa tra 2k e 16k; normalmente in un accesso al disco vengono scambiate molte pagine.

Magnetic
Disks



Read/Write
Head

Takes milliseconds to *seek* correct location on disk

Overview

- **Indirizzi virtuali**

- I programmi usano indirizzi virtuali
- L'intero spazio degli indirizzi virtuali è memorizzato nell'hard drive (memoria di massa)
- Essendo la memoria fisica (main memory) più piccola della memoria di massa, solo un sottoinsieme degli indirizzi virtuali è presente nella main memory
- gli indirizzi virtuali vengono tradotti in indirizzi fisici della main memory
- I dati che non vengono ritrovati nella main memory sono prelevati dalla memoria di massa (operazione molto dispendiosa in termini di tempo)

- **Memory Protection**

- Il Sistema operativo associa ad ogni programma un suo spazio di indirizzi virtuali
- Così un programma può non essere consapevole che altri programmi sono in esecuzione

Analogie tra Cache e memoria virtuale

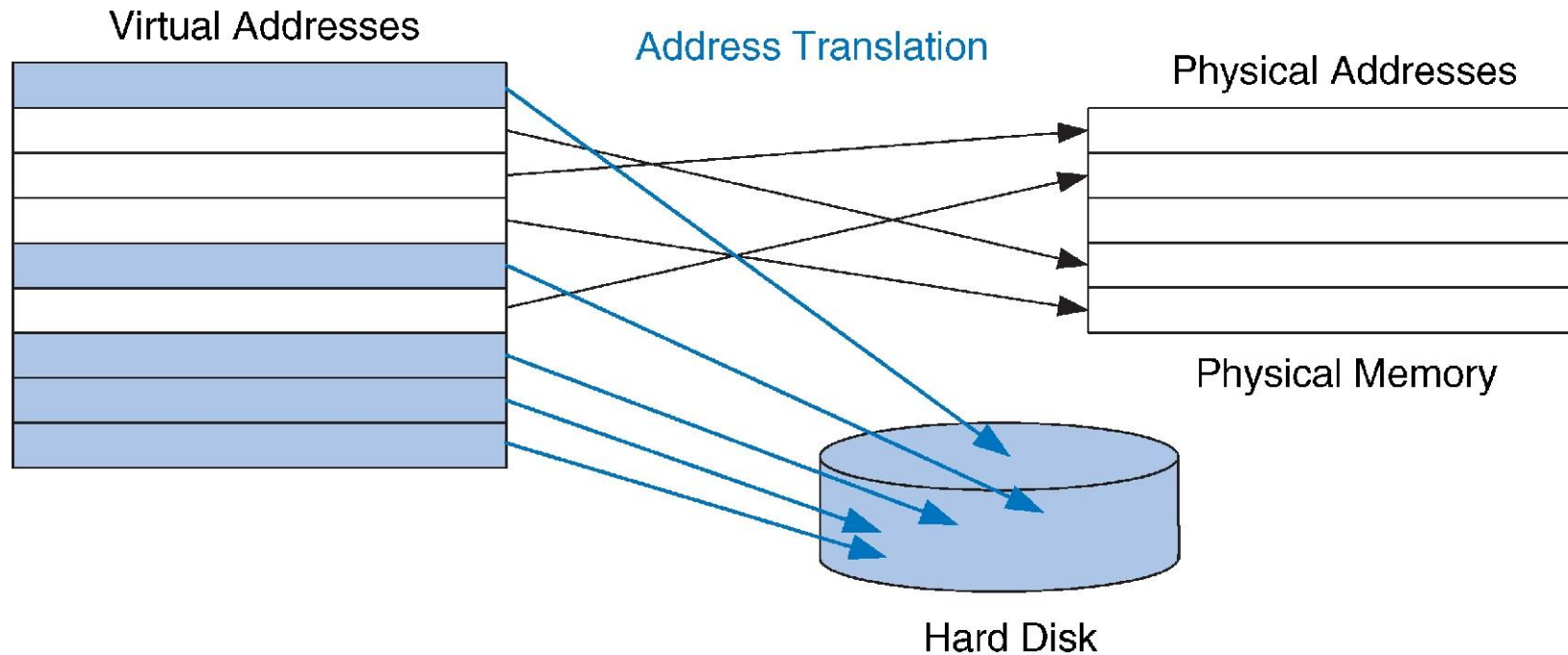
Cache	Virtual Memory
Block	Page
Block Size	Page Size
Block Offset	Page Offset
Miss	Page Fault
Tag	Virtual Page Number

La memoria fisica (main memory) agisce come cache per la memoria virtuale

Definizioni per la memoria virtuale

- **Page size:** dimensione dei segmenti di memoria trasferiti dalla memoria di massa alla main memory (2K-16K)
- **Address translation:** determinare l'indirizzo fisico nella main memory a partire da un indirizzo virtuale
- **Page table:** lookup table usata per tradurre gli indirizzi virtuali in indirizzi fisici

Indirizzi fisici e virtuali



- Data la grandezza delle pagine e la località spaziale e temporale durante l'esecuzione di un programma, la maggior parte degli accessi sono ritrovati nella main memory
- Ma i programmi sfruttano la maggior capacità della memoria virtuale

Esempio di memoria virtuale

System:

Virtual memory size: 2 GB = 2^{31} bytes

Physical memory size: 128 MB = 2^{27} bytes

Page size: 4 KB = 2^{12} bytes

Organization:

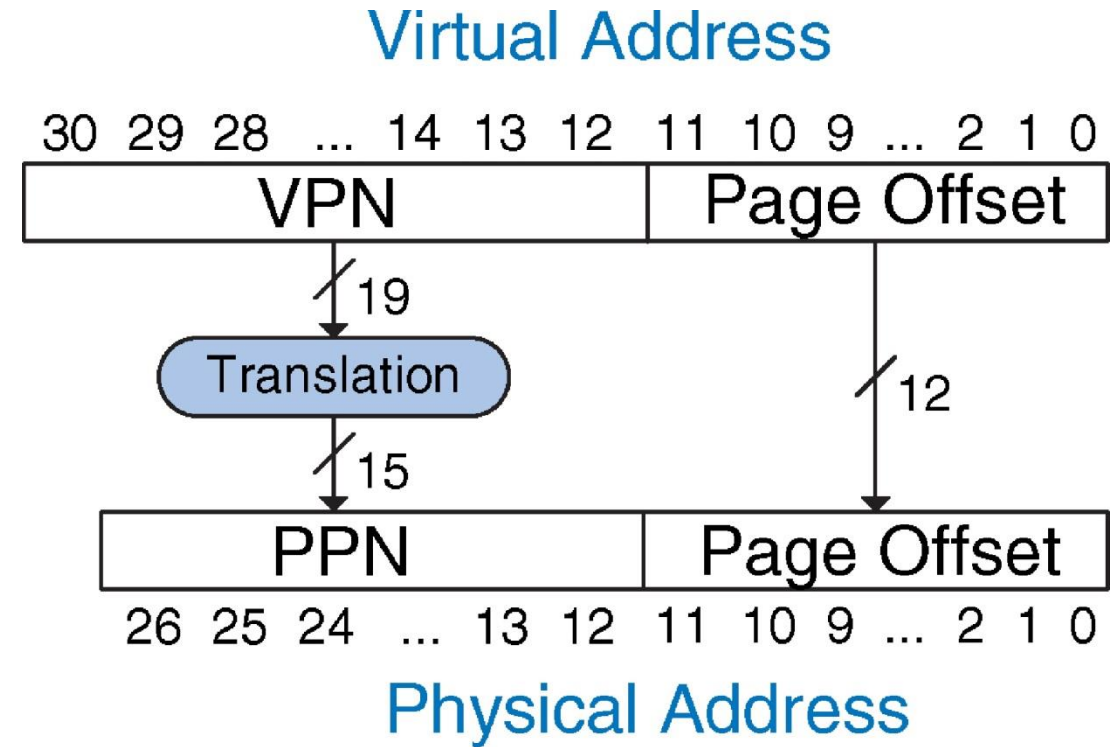
Virtual address: **31** bits

Physical address: **27** bits

Page offset: **12** bits

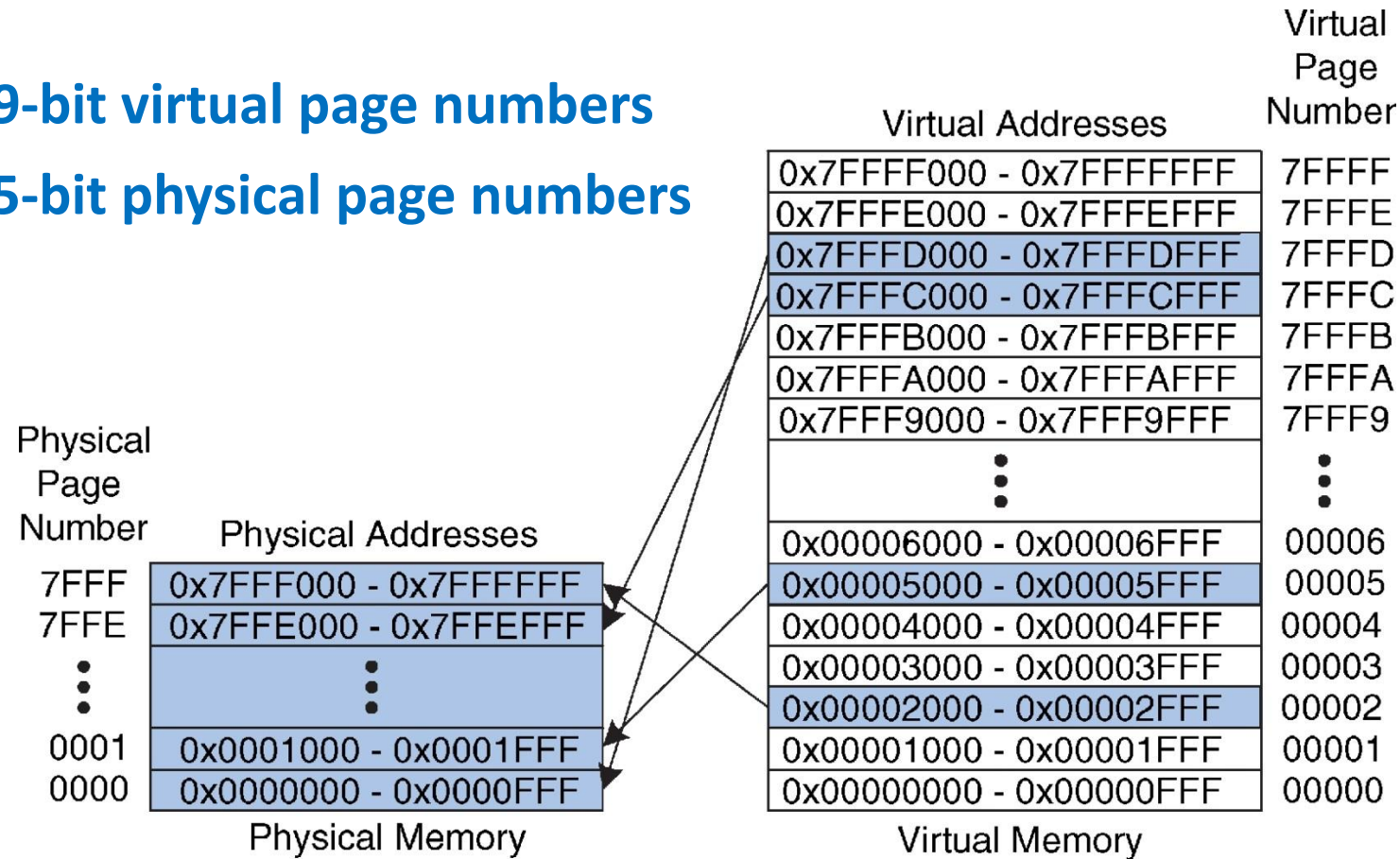
Virtual pages = $2^{31}/2^{12} = 2^{19}$ (VPN = **19** bits)

Physical pages = $2^{27}/2^{12} = 2^{15}$ (PPN = **15** bits)



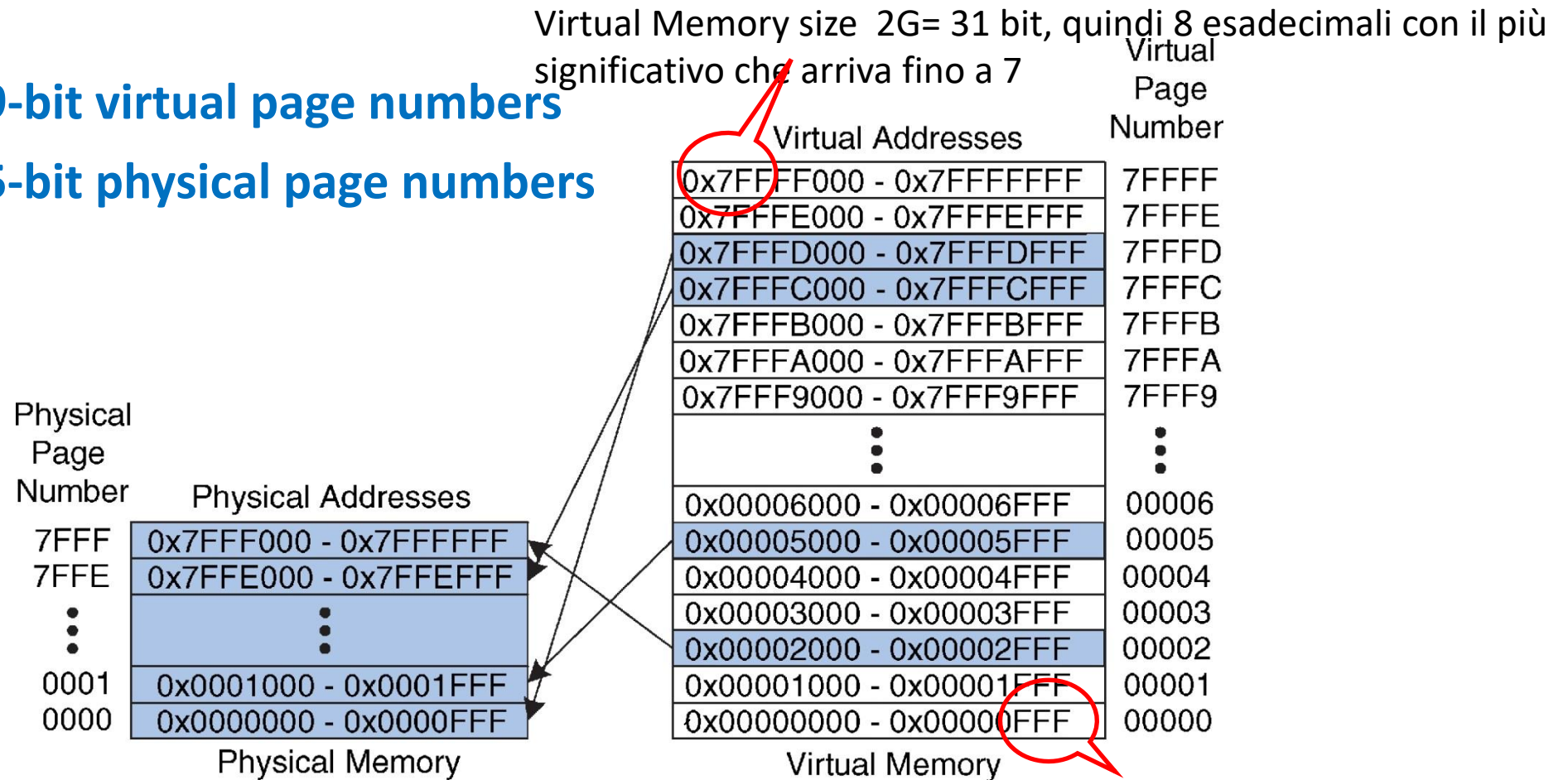
Esempio di memoria virtuale

- 19-bit virtual page numbers
- 15-bit physical page numbers



Esempio di memoria virtuale

- 19-bit virtual page numbers
- 15-bit physical page numbers



Page 4K= 12 bit=3 esadecimali

Esempio memoria virtuale

Quale è l'indirizzo fisico dell'indirizzo **0x247C**?

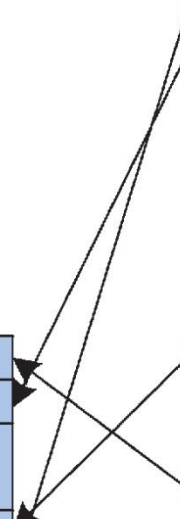
- VPN = **0x2**
- VPN 0x2 maps to PPN **0x7FFF**
- 12-bit page offset: **0x47C**
- Physical address = **0x7FFF47C**

Physical Page Number	Physical Addresses
7FFF	0x7FFF000 - 0x7FFFFF
7FFE	0x7FFE000 - 0x7FFEFFF
⋮	⋮
0001	0x0001000 - 0x0001FFF
0000	0x0000000 - 0x0000FFF

Physical Memory

Virtual Addresses	Virtual Page Number
0x7FFFF000 - 0x7FFFFFFF	7FFFF
0x7FFFE000 - 0x7FFFEFFF	7FFFE
0x7FFFD000 - 0x7FFFDFFF	7FFFD
0x7FFFC000 - 0x7FFFCFFF	7FFFC
0x7FFFB000 - 0x7FFFBFFF	7FFFB
0x7FFFA000 - 0x7FFFAFFF	7FFFA
0x7FFF9000 - 0x7FFF9FFF	7FFF9
⋮	⋮
0x00006000 - 0x00006FFF	00006
0x00005000 - 0x00005FFF	00005
0x00004000 - 0x00004FFF	00004
0x00003000 - 0x00003FFF	00003
0x00002000 - 0x00002FFF	00002
0x00001000 - 0x00001FFF	00001
0x00000000 - 0x00000FFF	00000

Virtual Memory



La Memory Management Unit (MMU)

Quello a cui fa riferimento il processore nell'elaborazione è detto indirizzo “**virtuale**” o logico ed ha come unica limitazione di essere entro il campo di indirizzamento massimo del processore.

Il compito di tradurre questo indirizzo in un indirizzo che punti ad una locazione effettivamente esistente nel sistema (**indirizzo fisico**) è demandato ad una struttura logica contenuta nel chip del processore, che si chiama **Memory Management Unit (MMU)** e che opera sotto il controllo software del sistema operativo.

Traduzione degli indirizzi

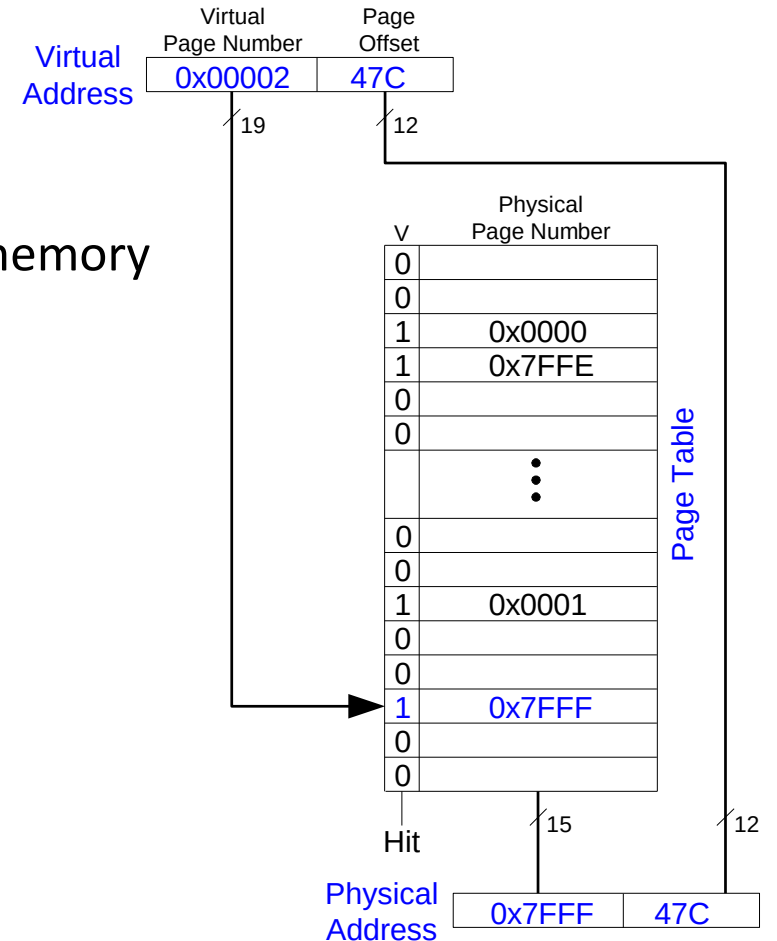
La memoria virtuale si comporta come una sorta di cache completamente associativa per il disco.

Tuttavia, se si gestisse la memoria virtuale come le cache, ossia tramite un comparatore sui bit più significativi per verificare la corrispondenza delle pagine, servirebbe un numero improponibile di comparatori, dato l'elevato numero di pagine in cui è organizzata la RAM.

In alternativa, il processo di traduzione degli indirizzi è gestito mediante **tabelle di paginazione**.

La page table

- Contiene un entry per ogni pagina virtuale
- Campi di una entry:
 - **Valid bit:** 1 se la pagina è presente nella main memory
 - **Physical page number:** dove la pagina è locata nella main memory



Esempio 1

Qual'è indirizzo fisico
dell'indirizzo virtuale

0x5F20?

- VPN = **5**
- Entry 5 in page table VPN
5 => physical page **1**
- Physical address: **0x1F20**

V	Physical Page Number
0	
0	
1	0x0000
1	0x7FFE
0	
0	
	⋮
0	
0	
1	0x0001
0	
0	
1	0x7FFF
0	
0	

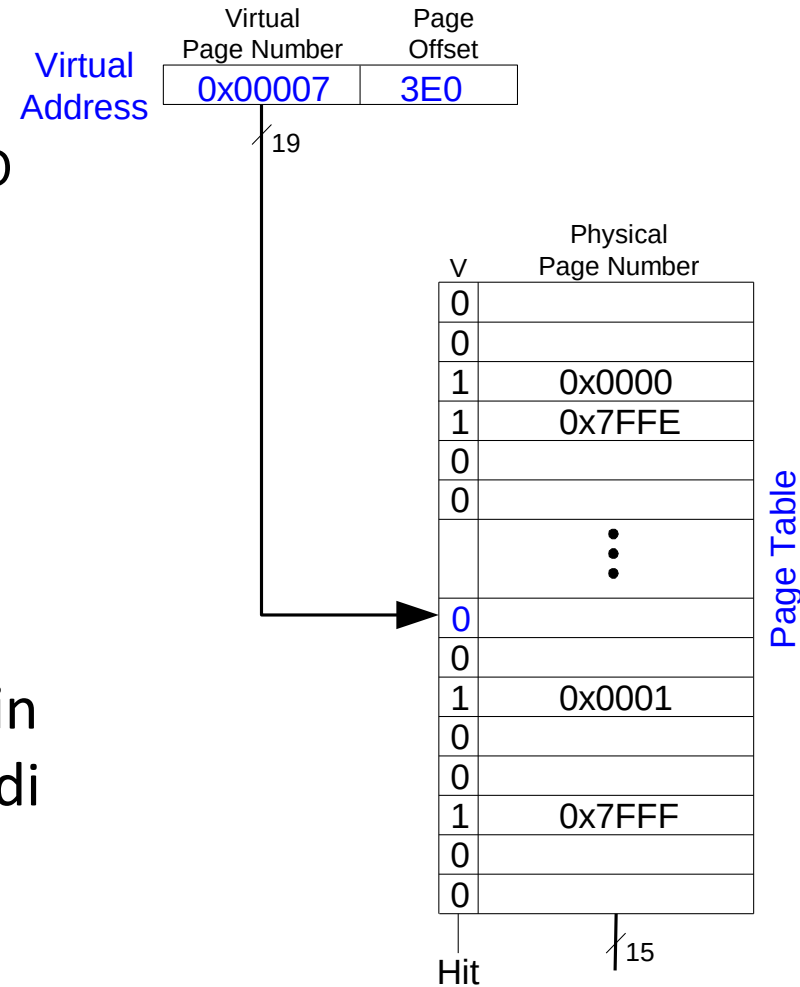
Page Table

Esempio 2

Quale è l'indirizzo fisico
dell'indirizzo virtuale

0x73E0?

- VPN = **7**
- L'entry 7 non è valida
- La pagina virtuale deve essere caricata nella main memory dalla memoria di massa



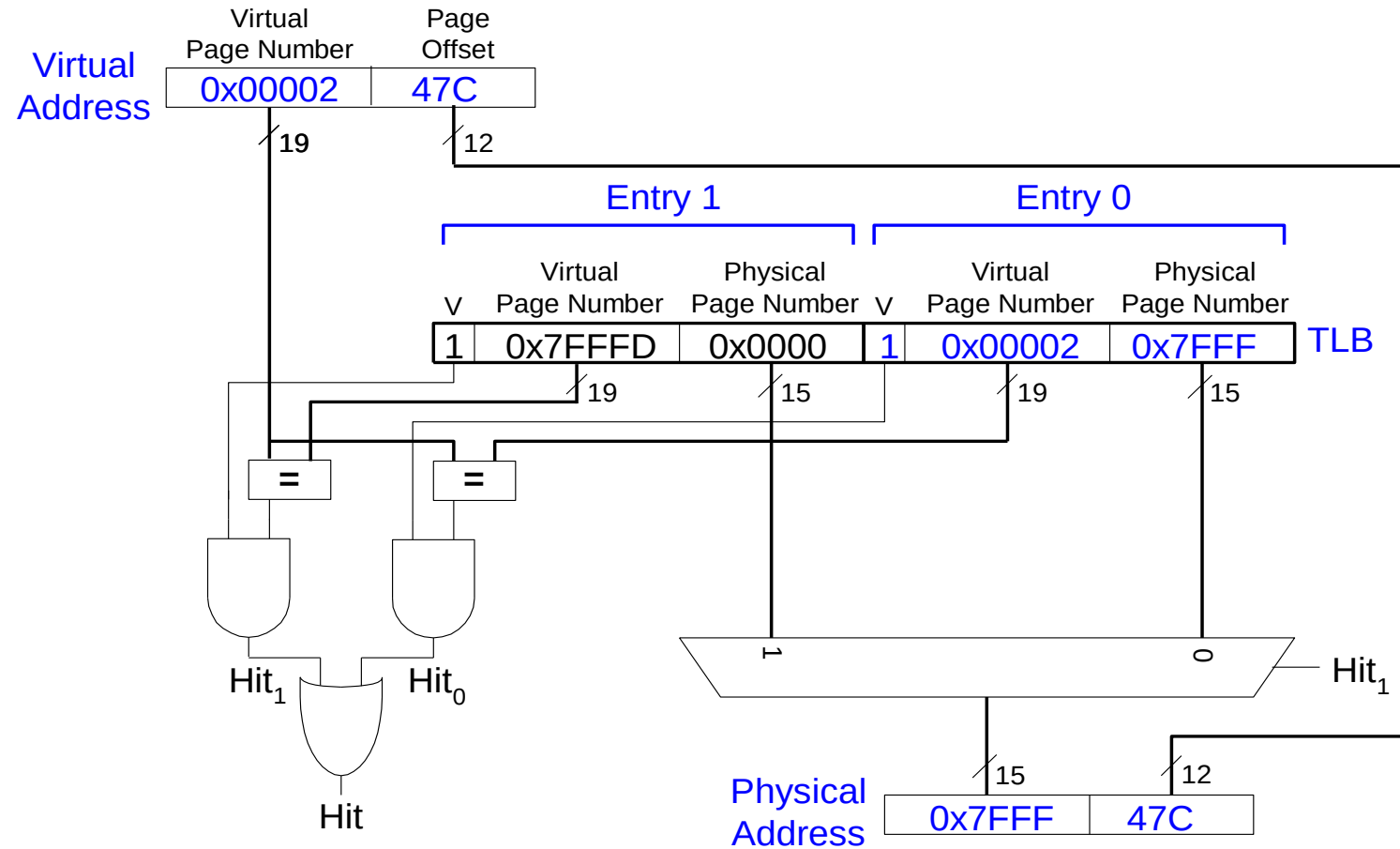
Caratteristiche della Page Table

- **La page table è solitamente voluminosa**
 - Di solito è essa stessa allocata nella main memory
- Le operazioni di load/store richiedono 2 accessi alla main memory:
 - Il primo per la traduzione (lettura della page table)
 - Il secondo è l'accesso vero e proprio al dato dopo la traduzione
- Questo dimezzerebbe le performance della main memory...

Translation Lookaside Buffer (TLB)

- Per migliorare le performance e far sì che la maggior parte delle operazioni di load e store avvenga con un solo accesso in memoria si usa una piccola cache dei traduzioni più recenti (TLB)
- TLB sfrutta il fatto che gli accessi alla page table godono di una elevata località temporale:
 - Essendo le dimensioni di una pagina relativamente grandi vi è un'alta probabilità che operazioni di load o store consecutive richiedano un accesso alla medesima pagina
- TLB
 - Veloce: accesso in < 1 cicli
 - Piccola: tipicamente 16 - 512 entry
 - Fully associative
 - Efficace: $> 99\%$ hit rates

Esempio di 2-Entry TLB



Esercizi sulle cache

Domanda: Si ipotizzi che il 30% delle istruzioni in un programma tipico effettuino un'operazione di scrittura o di lettura in RAM, e che la frequenza di successo di lettura in cache sia del 95% per le istruzioni e del 90% per i dati. Si supponga, inoltre, che la penalità di fallimento sia la stessa per operazioni di scrittura e operazioni di lettura. Si supponga che, in cicli di clock, una lettura diretta in memoria costi 10, una in cache costi 1 e una sia in cache che in memoria costi 16. Quale sarebbe il guadagno utilizzando la cache?

Esercizi sulle cache

Domanda: Si ipotizzi che il 30% delle istruzioni in un programma tipico effettuino un'operazione di scrittura o di lettura in RAM, e che la frequenza di successo di lettura in cache sia del 95% per le istruzioni e del 90% per i dati. Si supponga, inoltre, che la penalità di fallimento sia la stessa per operazioni di scrittura e operazioni di lettura. Si supponga che, in cicli di clock, una lettura diretta in memoria costi 10, una in cache costi 1 e una sia in cache che in memoria costi 16. Quale sarebbe il guadagno utilizzando la cache?

Date n istruzioni, costo senza cache è: $(1 + 0,3) n \cdot 10$

Costo con cache: $n (0,95 * 1 + 0,05 * 16) + 0,3 n (0,9 * 1 + 0,1 * 16)$

$$G = \frac{(1,3 * 10)n}{[(0,95 + 16 * 0,05) + (0,9 + 0,1 * 16)]n} = \frac{13}{2,5} = 5,2$$

Esercizi sulle cache

Domanda: Si ipotizzi che **4/5** delle istruzioni in un programma effettuino un'operazione di scrittura o di lettura in RAM, e che la frequenza di successo di lettura in cache per le istruzioni sia **2/3** di quella per i dati. Si supponga, inoltre, che la penalità di fallimento sia la stessa per operazioni di scrittura e operazioni di lettura. Si supponga che, in cicli di clock, una lettura diretta in memoria costi **10**, una in cache costi **1** e che una in entrambe (sia in memoria, che in cache) costi **16**. Quale deve essere il tasso di successo dei dati per avere un guadagno maggiore di **1**?

Date n istruzioni, costo senza cache è: $(1 + 4/5) * n * 10 = 18 n$

Costo con cache: $n (2/3 * x * 1 + (1 - 2/3 * x) 16) + 4/5 n (x * 1 + (1 - x) * 16)$

$$G = \frac{18}{[16 - 10x + 0,8(16 - 15x)]} = \frac{18}{28,8 - 22x} > 1$$

$$22x > 10,8$$

$$x > 0,49$$

Esercizi cache

Domanda 2 (Punti 5): Si consideri una memoria cache set-associativa da **2048** parole, suddivisa in **128** blocchi da **16** locazioni ciascuna, in cui si sceglie di avere **4** blocchi per insieme, ossia **32** insiemi. Sia data l'indirizzo **0x000244F4**, in quale insieme della cache essa sarà inserita e con quale etichetta? Analogamente, quali sono insieme ed etichetta se i blocchi per insieme fossero **2**?

0x000244F4 = 0000 0000 0000 0010 0100 0100 1111 0100

0000 0000 0000 0010 0100 0100 1111 0100

11 01 —————→ Parola nel set = 13

10011 —————→ Set = 19

0000 0000 0000 0010 0100 0 —————→ Etichetta = 72

Esercizi cache

Domanda 2 (Punti 5): Si consideri una memoria cache set-associativa da **2048** parole, suddivisa in **64** blocchi da **32** locazioni ciascuna, in cui si sceglie di avere **2** blocchi per insieme, ossia **32** insiemi. Sia data la parola **0x0000D33C**, in quale insieme della cache essa sarà inserita e con quale etichetta?

0x0000D33F = 0000 0000 0000 0000 1101 0011 0011 1100

011 11 → Parola nel set = 15

0011 0 → Set = 6

0000 0000 0000 0000 1101 → Etichetta = 13